

Programsko inženjerstvo ak.god 2024./2025
Sveučilište u Zagrebu
Fakultet elektrotehnike i računarstva
Skriptomat
Ciljevi projekta
Problematika zadatka
Potencijalna korist ovog projekta
Postojeća slična rješenja
Skup korisnika koji bi mogao biti zainteresiran za rješenje
Mogućnost prilagodbe rješenja
Opseg projektnog zadatka
Moguće nadogradnje projektnog zadatka
Funkcionalni zahtjevi
Ostali zahtjevi
 Zahtjevi za održavanje
Dionici
Obrasci uporabe
Dijagrami obrazaca uporabe
 1. Visokorazinski dijagram obrazaca uporabe cijelog sustava
 2. dijagram obrazaca uporabe za ključne značajke
 Dijagram obrazaca za registraciju, prijavu i odjavu
 Dijagram obrazaca za objavu skripte
 3. dijagram obrazaca uporabe za korisničke uloge
 4. dijagram obrazaca uporabe za osnovne poslovne procese
 5. dijagram obrazaca uporabe za kritične sustave i integracije
Opis obrazaca uporabe
 UC1 - Registracija korisnika
 UC1.1 - Odabir uloge
 UC2 - Prijava
 UC3 - Odjava
 UC4 - Slanje donacije
 UC5 - Primanje email obavijesti
 UC6 - Web chat
 UC7 - Ostavljanje recenzije
 UC8 - Slanje zahtjeva za objavljivanje skripte
 UC9 - Prihvatanje zahtjeva za objavljivanje skripte
 UC10 - Dodavanje skripte
 UC11 - Odobravanje novih moderatora
 UC12 - Upozoravanje i uklanjanje korisnika
 UC13 - Uređivanje i uklanjanje objava
 UC14 - Preuzimanje skripti
Sekvencijski dijagrami
 Dijagram za registraciju
 Dijagram objavljivanja skripte
Provjera uključenosti ključnih funkcionalnosti u obrasce uporabe
Arhitektura sustava
 Opis arhitekture
 Podsustavi
 Preslikavanje na radnu platformu
 Skalabilnost i budući razvoj
 Spremišta podataka
 Mrežni protokoli
 Globalni upravljački tok
 Sklopovsko-programski zahtjevi
 Obrazloženje odabira arhitekture
 Izbor arhitekture temeljen na principima oblikovanja
 Razmatrane alternative
Organizacija sustava na visokoj razini
Arhitektura sustava
 Baza podataka
 Datotečni sustav i pohrana medija
 Grafičko sučelje
Organizacija aplikacije

- Baza podataka
 - Opis tablica
 - USER
 - AUTHENTICATION
 - ROLE
 - COURSE
 - MATERIAL
 - COMMENT
 - RATING
 - SUBSCRIPTION
 - NOTIFICATION
 - DONATION
 - MESSAGE
 - MODERATION
 - Dijagram baze podataka
 - Relacijski model baze podataka
- Dijagram razreda
 - Dijagram razreda cijele aplikacije
 - Dijagram razreda korisnika i dokumenata
 - Dijagram razreda oauth2_provider-a
- Dinamičko ponašanje aplikacije
 - UML dijagrami stanja
 - Zahtjev za objavu skripte
 - UML dijagrami aktivnosti
 - Objava skripte
 - Dijagram komponenta
 - Dijagram razmještaja
- 6. Ispitivanje programskog rješenja
 - Ispitivanje komponenti
 - Struktura testova
 - Rezultati ispitivanja komponenti:
 - Ispitivanje sustava
 - 3. Selenium testovi - Sustavno testiranje (backend/tests/selenium_tests.py)
 - Rezultati ispitivanja sustava:
 - Sažetak svih testova:
 - Alati korišteni:
- 7. Korištene tehnologije i alati
 - Programski jezici**
 - Radni okviri i biblioteke**
 - Baza podataka**
 - Razvojni alati**
 - Alati za ispitivanje**
 - Alati za razmještaj**
 - Cloud platforma**
- Instalacija i pokretanje Skriptomat aplikacije
 - 1. Instalacija
 - Preduvjeti
 - Preuzimanje
 - Instalacija ovisnosti
 - 2. Postavke
 - Konfiguracijske datoteke
 - Postavke baze podataka
 - Pokretanje migracija
 - Popunjavanje inicijalnih podataka (seeding)
 - Kreiranje administratorskog korisnika
 - 3. Pokretanje aplikacije
 - Razvojno okruženje (Development)
 - Produkcijsko okruženje (Production)
 - 4. Upute za administratore
 - Pristup administratorskom sučelju
 - Administratorske mogućnosti
 - Redovito održavanje
 - Rješavanje problema
 - 5. Deploy na cloud platformu (Render.com primjer)

- Priprema repozitorija
- Postavljanje Backend servisa na Render
- Postavljanje Frontend servisa na Vercel
- Pokretanje aplikacije
- 6. Pristup aplikaciji na javnom poslužitelju
 - Pristup aplikaciji
 - Postupak korištenja
 - Pristup administratorskom sučelju
 - Ograničenja javnog poslužitelja
- Dodatne napomene
 - Sigurnosne prakse
 - Podrška
- Dnevnik sastajanja
- Tablica aktivnosti
- Dijagram pregleda promjena
- Ključni izazovi i rješenja

Programsko inženjerstvo ak.god 2024./2025

Sveučilište u Zagrebu

Fakultet elektrotehnike i računarstva

Skriptomat

Tim: <TG01.1>

JedanManje

Nastavnik: Vlado Sruk

JedanManje

Trenutno, studenti često koriste različite platforme za dijeljenje bilješki, poput društvenih mreža ili e-mailova, što može biti neučinkovito i kaotično. Također, takvim platformama nije svrha dijeljenje bilješki. Ovaj način dijeljenja informacija često rezultira gubitkom vremena i nemogućnošću pronalaska relevantnih materijala kada su najpotrebniji. Postoji jasna potreba za centraliziranom aplikacijom koja omogućuje jednostavno i brzo dijeljenje bilješki sortiranih prema kolegijima, semestrima, fakultetima itd., te ocjenjivanje kvalitete bilješki. Platforma „Skriptomat“ omogućuje dijeljenje vlastitih materijala među studentima čime se olakšava pristup relevantnim i ažurnim materijalima s predavanja. Studenti će imati brži i lakši pristup relevantnim materijalima za učenje te time uštedjeti vrijeme koje bi inače posvetili prikupljanju svih materijala potrebnih za učenje. Nadalje, poboljšat će se i kvaliteta bilješki sustavom ocjenjivanja koji omogućuje jednostavno identificiranje najkvalitetnijih materijala za učenje. Također, aplikacija potiče suradnju među studentima zato što međusobnom pomoći dolazi do boljeg razumijevanja gradiva, odnosno većeg akademskog uspjeha. Konačno, način na koji aplikacija svrstava bilješke, studentima će uštediti vrijeme koje je potrebno da pronađu potrebne materijale za vježbu. U nastavku se nalazi detaljan opis projekta.

Ciljevi projekta

Cilj projekta je stvaranje web platforme „Skriptomat“ koja omogućuje korisnicima dijeljenje bilješki s predavanja. Platforma funkcionira kao mini društvena mreža. Studenti imaju mogućnost pregledavanja i preuzimanja materijala te dodavanja vlastitih materijala. Uz svaki

objavljeni materijal moguće je ostaviti povratnu informaciju u obliku ocjene i komentara. Korisnici koji odaberu opciju „Moderator“ pregledavaju nove materijale i određuju njihovu točnost i prikladnost. Također, svaki korisnik može za bilo koju skriptu poslati donaciju pisatelju skripte u obliku „Buy me a coffee“ inicijative. Ovaj projekt osmišljen je kako bi se olakšao proces dijeljenja i pretraživanja bilješki među studentima, s naglaskom na organizaciju sadržaja i korisničke povratne informacije, što rezultira povećanjem kvalitete dostupnih materijala i poboljšanjem akademskih postignuća.

Problematika zadatka

Studenti često koriste različite platforme za dijeljenje bilješki, poput društvenih mreža ili e-mailova, što može biti neučinkovito i kaotično. Ovaj način dijeljenja informacija često rezultira gubitkom vremena i nemogućnošću pronalaska relevantnih materijala kada su najpotrebniji. Problem koji projekt „Skriptomat“ nastoji riješiti je pružiti jednostavno i brzo pronalaženje kvalitetnih bilješki.

Potencijalna korist ovog projekta

„Skriptomat“ pruža korist: * Studentima – omogućuje lakši pronalazak kvalitetnih materijala što dovodi do većeg akademskog uspjeha. Također, pruža korist i onim studentima koji su objavili dobru skriptu te za nju dobili donaciju.

Postojeća slična rješenja

Neke od aplikacija koje studenti koriste za pronalaženje materijala za učenje, ali to im nije glavna svrha:

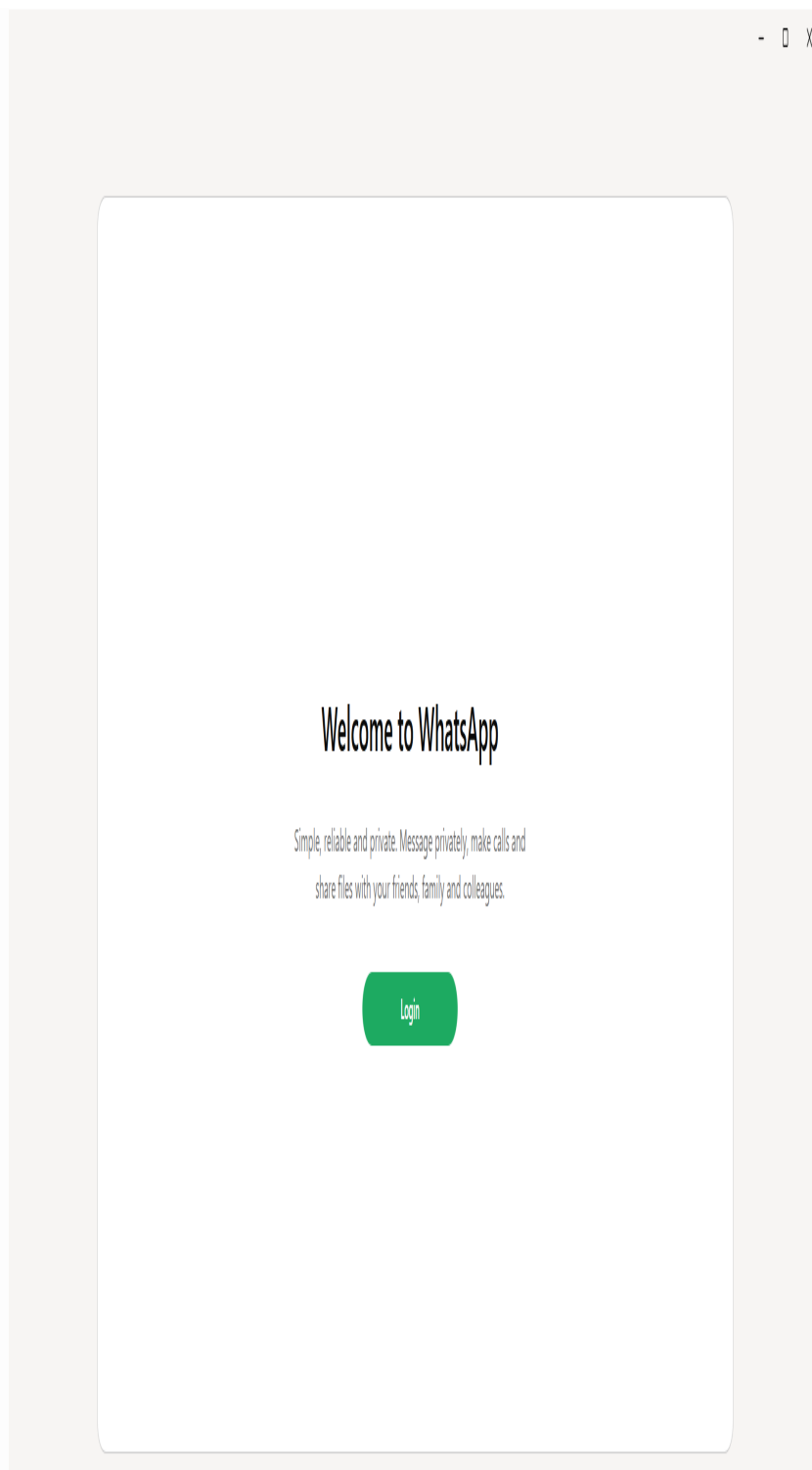
- **Discord** - aplikacija za kreiranje zajednica, najčešće korištena u svijetu videoigara, ali voljena među mnogim studentima radi mogućnosti objavljivanja bitnih materijala koja olakšavaju snalaženje s fakultetskim zadacima.



- **GitHub** - iako nije primarna namjena, studenti često u raznim neslužbenim GitHub repozitorijima objavljuju materijale za studiranje.



- **WhatsApp** - aplikacija za slanje poruka i poziva, a studenti ju koriste za dijeljenje korisnih materijala za učenje.



Skup korisnika koji bi mogao biti zainteresiran za rješenje

Skupina korisnika koja bi mogla biti zainteresirana za „Skriptomat“: * Studenti – koji žele brzo pronaći kvalitetne materijale za učenje.

Mogućnost prilagodbe rješenja

“Skriptomat” nudi fleksibilnu prilagodbu različitim korisnicima i njihovim potrebama. Naprimjer: * Dizajn prilagođen web i mobilnim uređajima omogućuje jednostavan pristup materijalima s predavanja na različitim platformama. * Organizacija materijala prema fakultetima, godinama, kolegijima kako bi korisnici mogli prilagoditi pregled materijala vlastitim potrebama.

Opseg projektnog zadatka

Projekt “Skriptomat” obuhvaća sljedeći opseg: * Razvoj funkcionalnosti za registraciju korisnika (Moderator/Student). * Razvoj sustava za pohranu, organizaciju i dijeljenje materijala s predavanja. * Integriranje postojećeg rješenja za web chat pomoću kojeg studenti mogu razgovarati međusobno ili kontaktirati moderatore. * Slanje donacija autoru skripte korištenjem vanjskog servisa. * Uloga administratora za upravljanje korisnicima, odobravanje novih moderatora i nadgledanje rada sustava.

Moguće nadogradnje projektnog zadatka

Nadogradnje projekta “Skriptomat” mogu biti: * Dodatni formati bilješki. * Dodavanje materijala s fakulteta diljem Hrvatske. * Dodavanje fakulteta u drugim državama povezanih s fakultetima u Hrvatskoj putem projekata (Erasmus...).

Funkcionalni zahtjevi

ID zahtjeva	Opis	Prioritet	Izvor	Kriteriji prihvatanja
F-001	Sustav omogućuje registraciju korisnika kao moderatora ili regularnih korisnika.	Visok	Zahtjev dionika	Korisnici se na platformu mogu registrirati e-mail adresom i lozinkom korištenjem vanjskih servisa za autentifikaciju. Uz vlastite podatke, korisnici moraju odabrati i ulogu: moderatori ili regularni korisnici.
F-002	Sustav omogućuje prijavu korisnika.	Visok	Zahtjev dionika	Registrirani korisnici se kasnije mogu prijaviti na svoj račun koristeći podatke kojima su se registrirali.
F-003	Sustav omogućuje regularnim korisnicima objavljivanje materijala.	Visok	Zahtjev dionika	Korisnik može objaviti svoju skriptu. Svaki materijal mora sadržavati osnovne podatke: godinu nastanka, naziv kolegija i naslov.
F-004	Sustav omogućuje regularnim korisnicima recenziranje i ocjenjivanje materijala.	Visok	Zahtjev dionika	Korisnik može uz svaki objavljeni materijal ostaviti povratnu informaciju u obliku ocjene „upvote“ ili „downvote“ i komentara.
F-005	Sustav omogućuje moderatorima provjeru validnosti objavljenih materijala.	Visok	Zahtjev dionika	Moderatori mogu pregledati nove materijale i određivati njihovu točnost i prikladnost te imaju mogućnost označiti sadržaj kao neprimjeren ili odobriti objavu. Oni pregledavaju sadržaj prije objavljivanja.

ID zahtjeva	Opis	Prioritet	Izvor Zahtjev dionika	Kriterij prihvatanja
F-006	Sustav omogućuje regularnim korisnicima slanje donacija.	Visok		Svaki korisnik može za bilo koju skriptu poslati donaciju pisatelju skripte u obliku „Buy me a coffee” inicijative.
F-007	Sustav ima postojeće rješenje za web chat pomoću kojeg korisnici razgovaraju.	Visok	Zahtjev dionika	Unutar aplikacije integrirano je postojeće rješenje za web chat pomoću kojeg studenti mogu razgovarati međusobno ili kontaktirati moderatore.
F-008	Sustav omogućuje administratorima odobravanje novih moderatora, upozoravanje i uklanjanje korisnika.	Visok	Zahtjev dionika	U sustavu postoje administratori koji upravljaju korisnicima, odobravaju nove moderatore.
F-009	Sustav omogućuje administratorima uklanjanje i uređivanje objava.	Visok	Zahtjev dionika	Administratori nadgledaju rad sustava.
F-010	Sustav omogućuje objavu materijala u pdf formatu.	Visok	Zahtjev dionika	Materijali se objavljuju u pdf formatu.
F-011	Sustav omogućuje dobivanje obavijesti za aktivnosti na platformi.	Visok	Zahtjev dionika	Kada moderator potvrdi skriptu, korisniku koji je objavio skriptu dolazi potvrda/obavijest o prihvatanju skripte.
F-012	Sustav omogućuje regularnim korisnicima preuzimanje materijala.	Visok	Zahtjev dionika	Korisnik može preuzeti objavljene materijale drugog korisnika ako je autor skripte dopustio preuzimanje.

Ostali zahtjevi

Zahtjevi za održavanje

ID zahtjeva	Opis	Prioritet
NF-1.1	Objava materijala izvršava se unutar maksimalno dvije minute.	Visok
NF-1.2	Sustav koristi HTTPS za prijenos podataka.	Visok
NF-1.3	Registracija i prijava korisnika ostvarena je korištenjem vanjskih servisa za autentifikaciju (OAuth 2.0).	Visok
NF-1.4	Veličina pdf datoteke materijala ne smije biti veća od 50 MB.	Visok
NF-1.5	Uplate donacija omogućeno putem integracije s vanjskim servisima (PayPal, kreditne kartice).	Visok
NF-1.6	Za implementaciju funkcionalnosti razgovora treba integrirati postojeća rješenja kao što je npr. FreeChat.	Visok
NF-1.7	Korisničko sučelje mora biti intuitivno za korištenje	Visok
NF-1.8	Sustav mora biti dostupan korisnicima 95% vremena.	Visok

Dionici

1. Regularni korisnici - Osobe koje mogu objavljivati materijale, recenzirati i ocjenjivati te

ostavljati donacije.

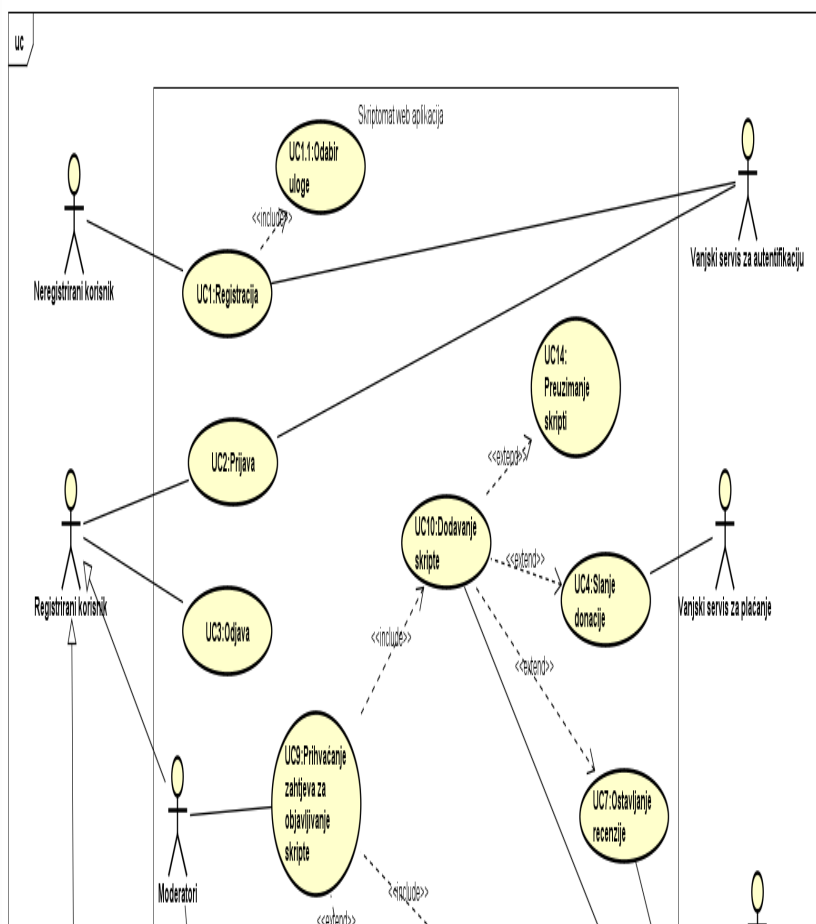
2. Moderatori - Osobe koje će pregledati nove materijale i određivati njihovu točnost i prikladnost te imaju mogućnost označiti sadržaj kao neprimjeren ili odobriti objavu.
3. Administratori - Osobe zadužene za upravljanje korisnicima, odobravanje novih moderatora i nadgledanje rada sustava.
4. Razvojni tim - Programeri i dizajneri zaduženi za izradu, razvoj i održavanje aplikacije. Voditelj projekta, stručnjak za testiranje, voditelj tehničkog razvoja, backend developer, frontend developer, UI/UX dizajner.
5. Vanjski servisi - API servisi koji komuniciraju sa sustavom: OAuth 2.0 (autentifikacija), PayPal (plaćanje), FreeChat (razgovor).
6. Naručitelj projekta - Pokretač projekta, definira osnovne ciljeve te odobrava konačne specifikacije i značajke sustava.

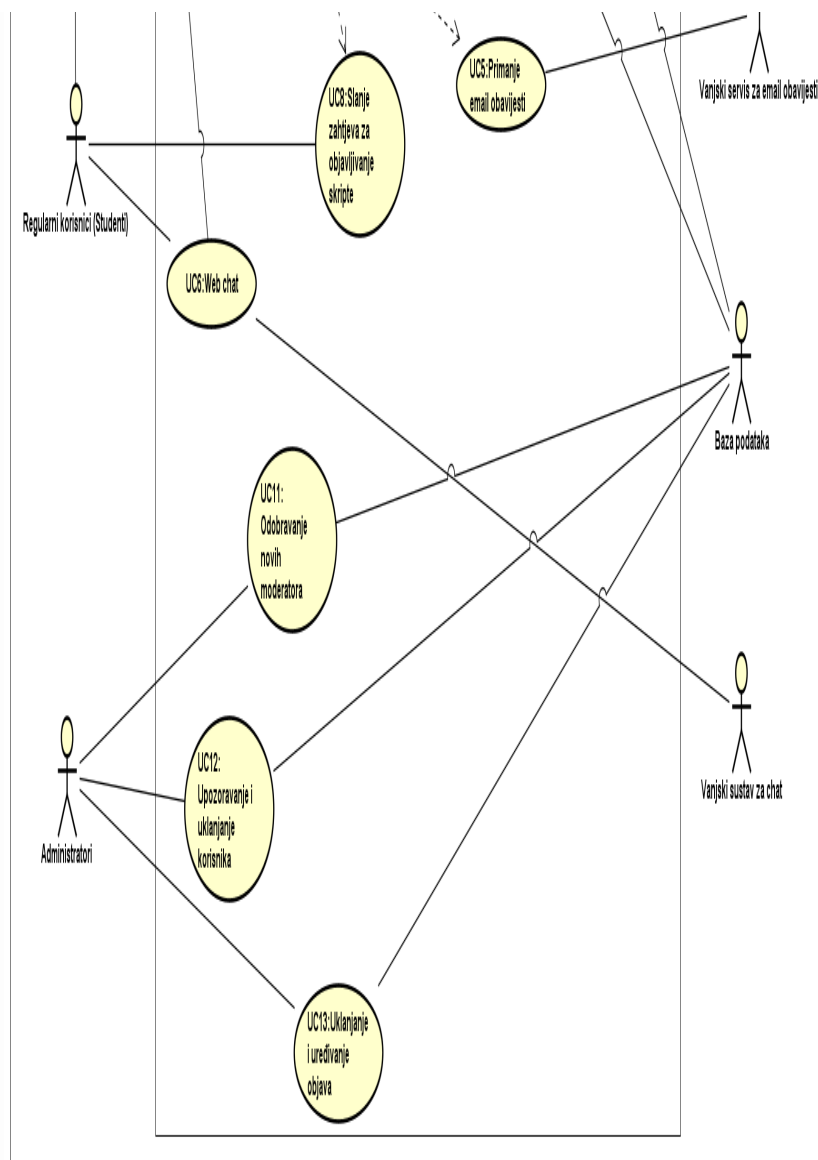
Aktori i njihovi funkcionalni zahtjevi: * **A-1 Neregistrirani korisnik** može: F-001 * **A-2 Moderator (Registrirani korisnik)** može: F-002, F-005, F-007, F-011 * **A-3 Student (Registrirani korisnik)** može: F-002, F-003, F-004, F-006, F-007, F-010, F-011, F-012 * **A-4 Administrator** može: F-008, F-009

Obrasci uporabe

Dijagrami obrazaca uporabe

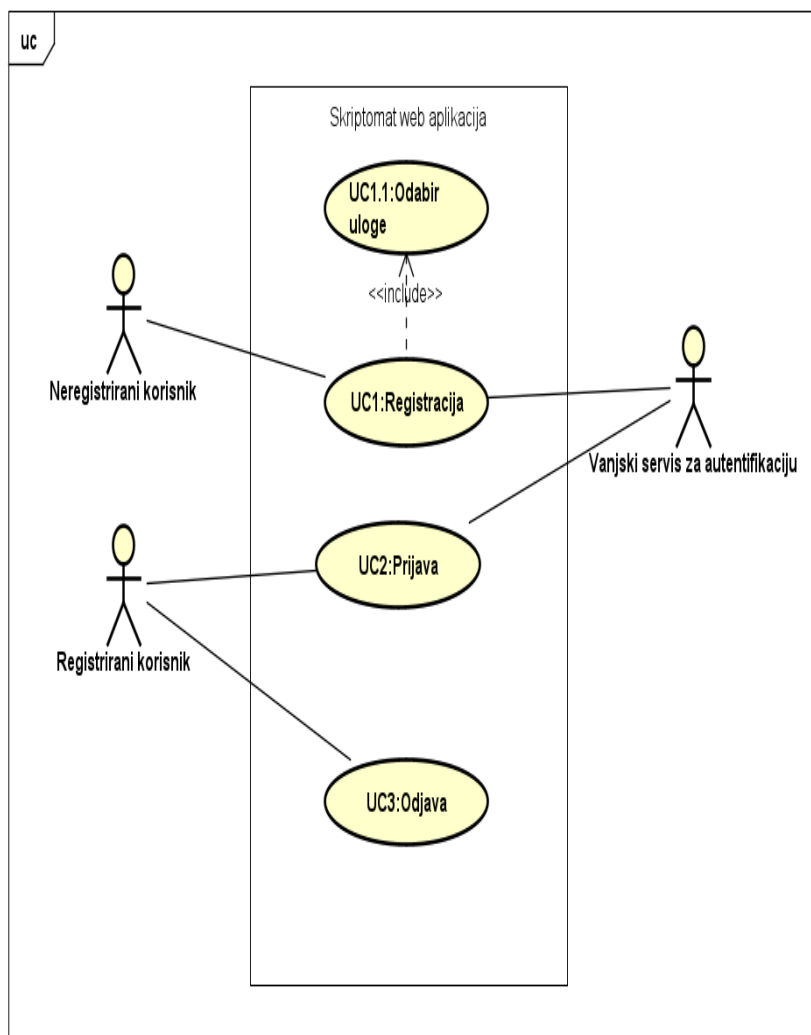
1. Visokorazinski dijagram obrazaca uporabe cijelog sustava



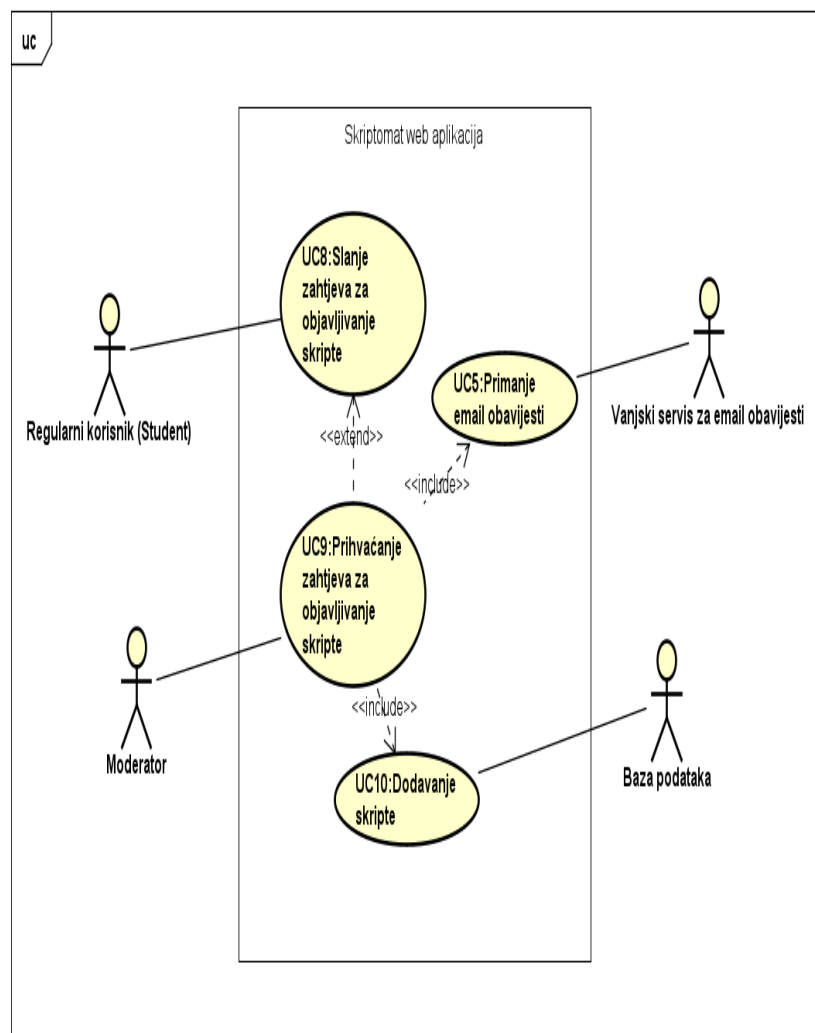


2. dijagram obrazaca uporabe za ključne značajke

Dijagram obrazaca za registraciju, prijavu i odjavu

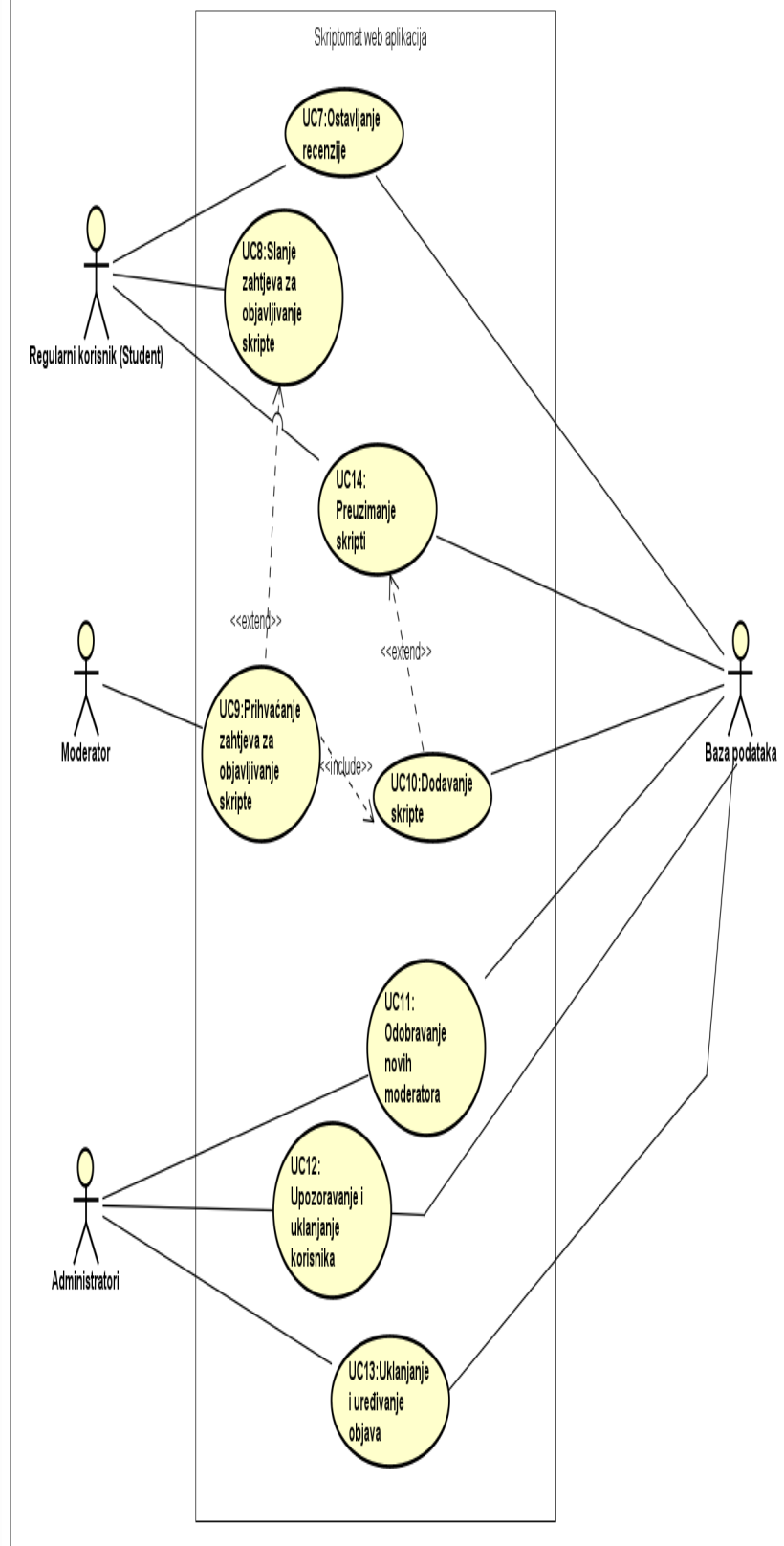


Dijagram obrazaca za objavu skripte

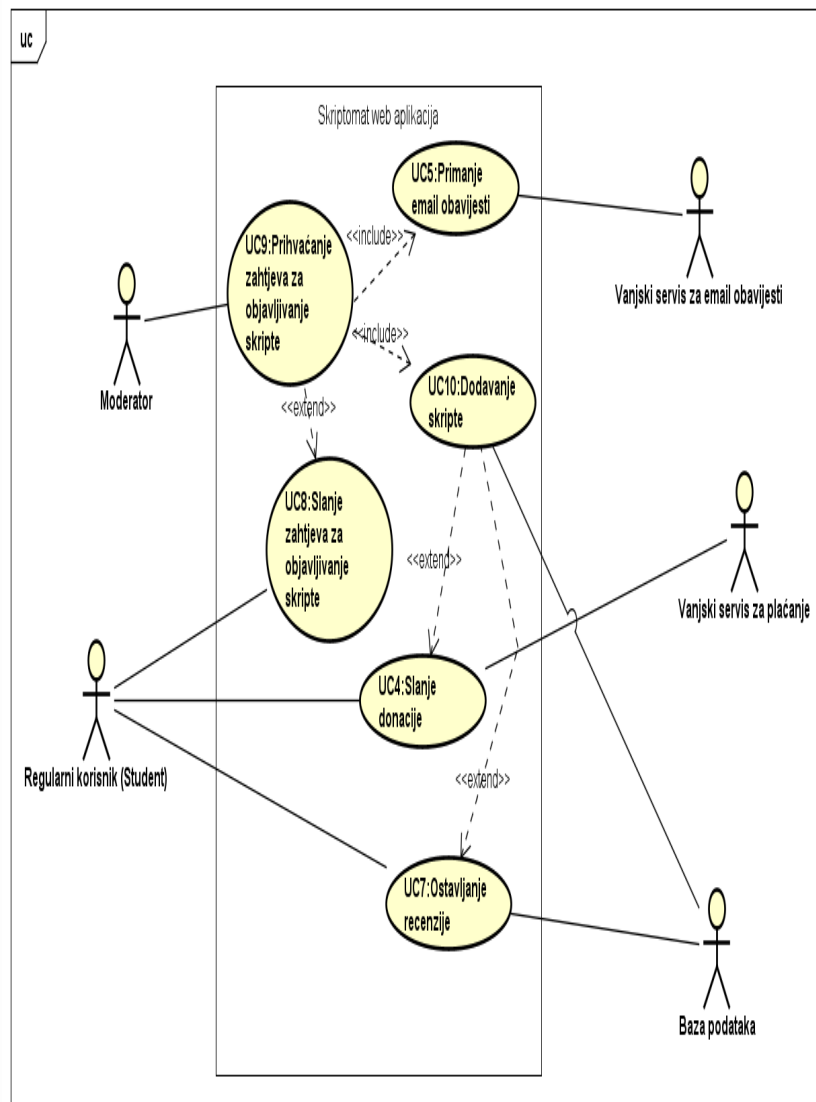


3. dijagram obrazaca uporabe za korisničke uloge

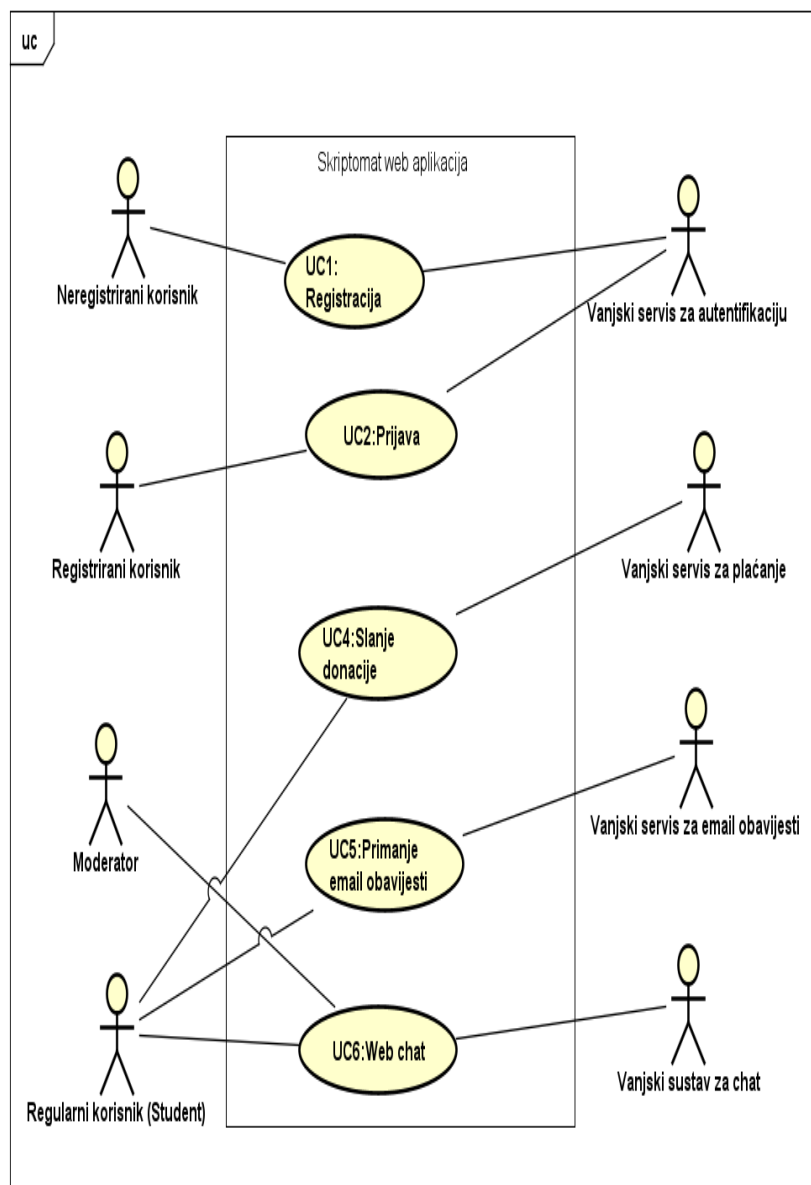
uc



4. dijagram obrazaca uporabe za osnovne poslovne procese



5. dijagram obrazaca uporabe za kritične sustave i integracije



Opis obrazaca uporabe

UC1 - Registracija korisnika

- Glavni sudionik: neregistrirani korisnik
- Cilj: registrirati se u sustav
- Sudionici: neregistrirani korisnik, vanjski servis za autentifikaciju
- Preduvjet: korisnik nema napravljen račun
- Opis osnovnog tijeka:

1. Korisnik odabire opciju "Registracija". (F-001)
2. Sustav prikazuje obrazac za registraciju.
3. Korisnik unosi email, lozinku.
4. Sustav šalje podatke vanjskom servisu za autentifikaciju.
5. Sustav kreira korisnički profil.

- Opis mogućih odstupanja:

1. Neispravan email.
2. Email već postoji - sustav predlaže prijavu.

UC1.1 - Odabir uloge

- Glavni sudionik: neregistrirani korisnik
- Cilj: odabrati ulogu prije registracije
- Sudionici: neregistrirani korisnik, vanjski servis za autentifikaciju
- Preduvjet: korisnik je započeo proces registracije
- Opis osnovnog tijeka:

1. Sustav pita korisnika da odabere ulogu. (F-001)
2. Korisnik odabire moderator/regularni korisnik.
3. Sustav sprema odabranu ulogu i proslijeđuje je dalje u registraciju.

- Opis mogućih odstupanja:

1. Korisnik odustaje - registracija se prekida.

UC2 - Prijava

- Glavni sudionik: registrirani korisnik
- Cilj: prijaviti se u sustav
- Sudionici: registrirani korisnik, vanjski servis za autentifikaciju
- Preduvjet: korisnik je registriran
- Opis osnovnog tijeka:

1. Korisnik dolazi na stranicu i odabire servis na kojem ima otvoren račun.

- Opis mogućih odstupanja:

1. Pogrešna lozinka - sustav javlja grešku.

UC3 - Odjava

- Glavni sudionik: registrirani korisnik
- Cilj: odjava iz sustava
- Sudionici: registrirani korisnik
- Preduvjet: korisnik je prijavljen
- Opis osnovnog tijeka:

1. Korisnik klikne "Odjava".
2. Korisnik je preusmjeren na početnu stranicu.

- Opis mogućih odstupanja:

1. U slučaju da se korisnik ne odjavi, ostat će ulogiran.

UC4 - Slanje donacije

- Glavni sudionik: regularni korisnik
- Cilj: donirati neki iznos autoru skripte
- Sudionici: regularni korisnik, vanjski servis za plaćanje
- Preduvjet: korisnik je prijavljen, skripta postoji
- Opis osnovnog tijeka:

1. Korisnik odabire "Buy me a coffee".
2. Sustav šalje zahtjev servisu za plaćanje.
3. Korisnik unosi podatke za uplatu.
4. Servis provodi transakciju.
5. Sustav prikazuje potvrdu o donaciji.

- Opis mogućih odstupanja:

1. Plaćanje odbijeno - korisnik dobiva upozorenje.

UC5 - Primanje email obavijesti

- Glavni sudionik: registrirani korisnik
- Cilj: dobiti obavijest o aktivnostima (potvrda skripte)
- Sudionici: registrirani korisnik, vanjski servis za email obavijesti
- Preduvjet: moderator je odobrio skriptu
- Opis osnovnog tijeka:

1. Moderator odobrava skriptu.
2. Sustav generira obavijest.
3. Obavijest se šalje email servisu.
4. Korisnik prima obavijest.

- Opis mogućih odstupanja:

1. Email servis nedostupan - slanje se ponavlja kasnije.

UC6 - Web chat

- Glavni sudionik: registrirani korisnik
- Cilj: razgovarati s drugim korisnicima
- Sudionici: registrirani korisnik, vanjski sustav za chat
- Preduvjet: korisnik je prijavljen
- Opis osnovnog tijeka:

1. Korisnik otvara chat.
2. Sustav povezuje korisnika s chat servisom.
3. Korisnik šalje i prima poruke.

- Opis mogućih odstupanja:

1. Chat servis nedostupan.

UC7 - Ostavljanje recenzije

- Glavni sudionik: regularni korisnik
- Cilj: ocijeniti ili komentirati skriptu
- Sudionici: regularni korisnik, baza podataka
- Preduvjet: skripta postoji
- Opis osnovnog tijeka:

1. Korisnik otvara skriptu.
2. Odabire “upvote”, “downvote”, ostavljanje komentara.
3. Unosi komentar i ocjenu.
4. Sustav sprema recenziju.

- Opis mogućih odstupanja:

1. Korisnik nije prijavljen - sustav traži login.

UC8 - Slanje zahtjeva za objavljivanje skripte

- Glavni sudionik: regularni korisnik
- Cilj: poslati zahtjev za objavu skripte
- Sudionici: regularni korisnik, moderator
- Preduvjet: korisnik je prijavljen, skripta u pdf formatu
- Opis osnovnog tijeka:

1. Korisnik odabire skriptu za objavu.
2. Odabire pdf datoteku.
3. Unosi podatke (godina, kolegij, naslov).
4. Sustav sprema skriptu kao zahtjev.
5. Moderatoru se prikazuje novi zahtjev.

- Opis mogućih odstupanja:

1. Datoteka nije pdf - greška.
2. Datoteka prevelika - odbijeno.

UC9 - Prihvaćanje zahtjeva za objavljivanje skripte

- Glavni sudionik: moderator
 - Cilj: pregledati i odobriti skriptu
 - Sudionici: regularni korisnik, moderator
 - Preduvjet: korisnik je poslao zahtjev
 - Opis osnovnog tijeka:
1. Moderator otvara listu zahtjeva.
 2. Pregledava skriptu.
 3. Može označiti kao prihvatljivo ili neprimjereno.
 4. Ako prihvati - skripta postaje vidljiva.
 5. Sustav šalje email autoru.
- Opis mogućih odstupanja:
1. Skripta neprimjerena - moderator odbija.

UC10 - Dodavanje skripte

- Glavni sudionik: regularni korisnik
 - Cilj: objaviti skriptu
 - Sudionici: regularni korisnik, moderator, baza podataka
 - Preduvjet: zahtjev za objavu skripte prihvaćen
 - Opis osnovnog tijeka:
1. Kombiniranje UC8 i UC9.
- Opis mogućih odstupanja:
1. Korisnik ne unese sve podatke potrebne za objavu.

UC11 - Odobravanje novih moderatora

- Glavni sudionik: administrator
 - Cilj: odobriti korisniku moderatorska prava
 - Sudionici: administrator, moderator, baza podataka
 - Preduvjet: korisnik je registriran
 - Opis osnovnog tijeka:
1. Administrator otvara listu korisnika.
 2. Odabire korisnika za unapređenje.
 3. Odobrava moderatorsku ulogu.
 4. Sustav ažurira korisničke podatke.

UC12 - Upozoravanje i uklanjanje korisnika

- Glavni sudionik: administrator
 - Cilj: upozoriti ili ukloniti korisnika
 - Sudionici: administrator, korisnik, baza podataka
 - Preduvjet: korisnik postoji u sustavu
 - Opis osnovnog tijeka:
1. Administrator odabire korisnika.
 2. Odlučuje između upozorenja ili brisanja.
 3. Sustav šalje upozorenje ili briše korisnika.

UC13 - Uređivanje i uklanjanje objava

- Glavni sudionik: administrator
- Cilj: ukloniti ili urediti objavu

- Sudionici: administrator, baza podataka
- Preduvjet: objava postoji
- Opis osnovnog tijeka:

1. Administrator otvara objavu.
2. Odabire uređivanje ili brisanje.
3. Sustav ažurira ili uklanja objavu.

UC14 - Preuzimanje skripti

- Glavni sudionik: regularni korisnik
- Cilj: preuzeti dostupnu skriptu
- Sudionici: regularni korisnik, baza podataka
- Preduvjet: objava postoji i autor je omogućio preuzimanje
- Opis osnovnog tijeka:

1. Korisnik odabire skriptu.
2. Sustav provjerava prava pristupa.
3. Korisnik preuzima datoteku.

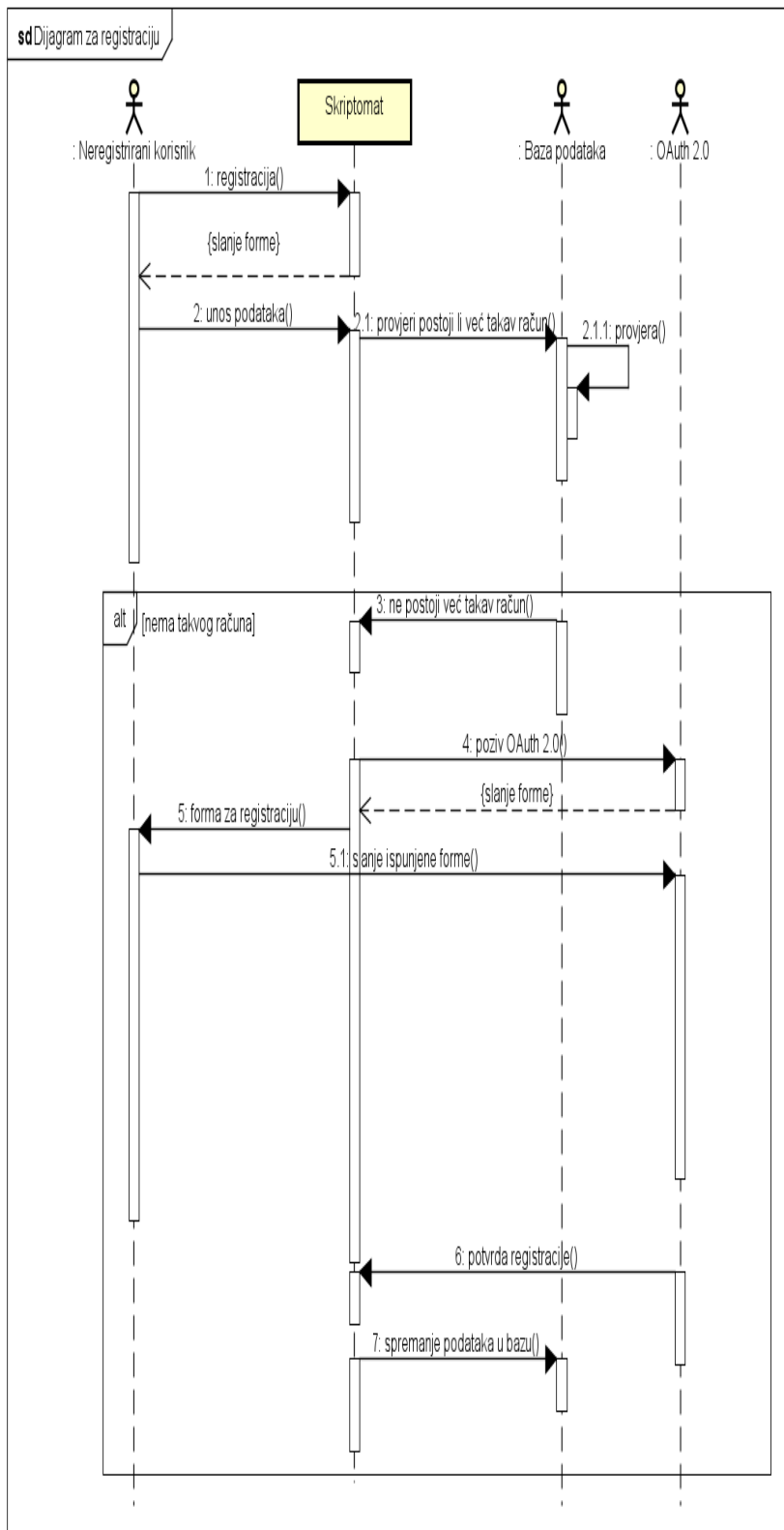
- Opis mogućih odstupanja:

1. Autor nije omogućio preuzimanje - greška.

Sekvencijski dijagrami

Dijagram za registraciju

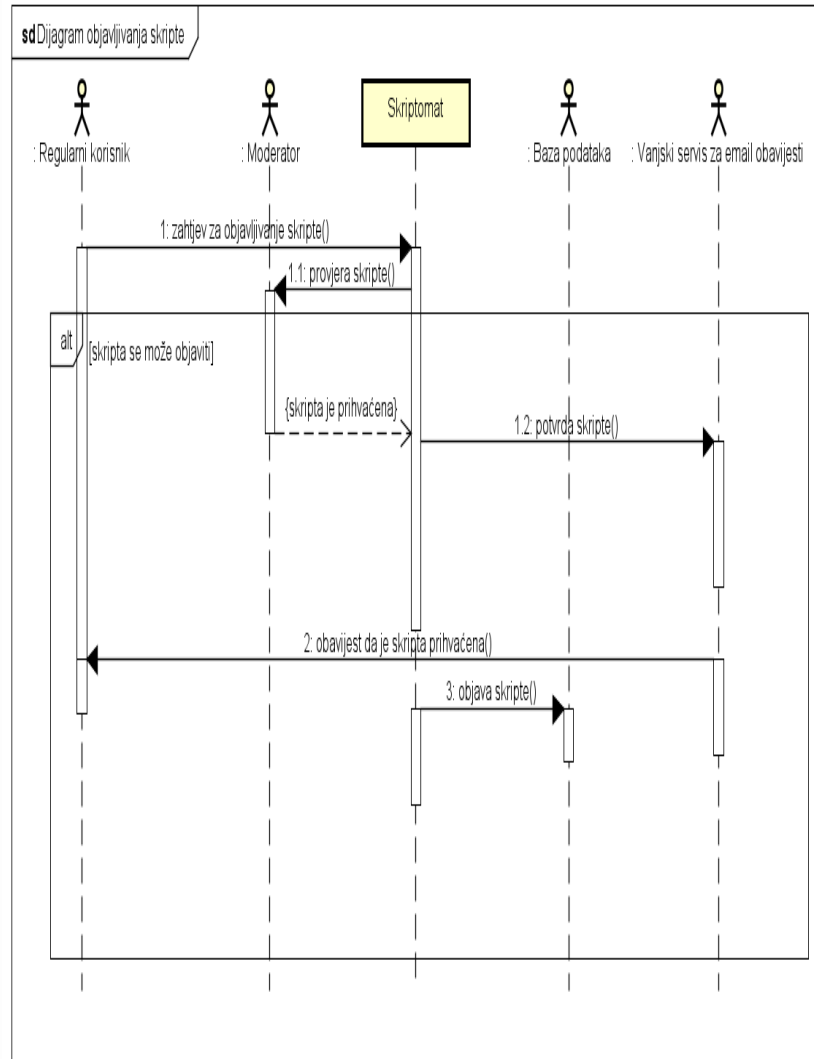
Ovaj sekvencijski dijagram prikazuje tok registracije novog korisnika u sustavu Skriptomat. Dijagram opisuje interakciju između četiri sudionika: Neregistrirani korisnik, Skriptomat web aplikacija, Baza podataka i OAuth 2.0 servis. Opisuje tijek dodavanja korisnika u bazu kad on ne postoji. Proces započinje kada neregistrirani korisnik odabere opciju registracije. Sustav mu prikazuje formu za unos registracijskih podataka. Korisnik unosi svoje podatke u formu, a Skriptomat zatim šalje zahtjev prema bazi kako bi provjerio postoji li već račun s tim podacima. Ako baza ne pronađe postojeći račun, sustav nastavlja dalje. Nakon što je utvrđeno da takav račun ne postoji, Skriptomat inicira provjeru identiteta putem OAuth 2.0 servisa. Korisnik ispunjava formu, a rezultat se vraća u Skriptomat. Skriptomat prikazuje korisniku završnu registracijsku formu, a on šalje ispunjene podatke natrag sustavu. Sustav zatim šalje korisniku obavijest o uspješnoj registraciji. Konačno, Skriptomat pohranjuje sve korisnikove podatke u bazu podataka, čime je registracija završena.



Dijagram objavljivanja skripte

Ovaj sekvencijski dijagram prikazuje tok objavljivanja skripte na web aplikaciju Skriptomat.

Dijagram opisuje interakciju između pet sudionika: Regularni korisnik, Moderator, Baza podataka i Vanjski servis za email obavijesti. Opisuje tijek dodavanja skripte u bazu kad je ona prikladna za objavu. Proces započinje kada regularni korisnik pošalje zahtjev za objavu skripte. Sustav zahtjev proslijeđuje moderatoru. Moderator označi da je skripta primjerena za objavu i prihvatanje se šalje vanjskom servisu za email obavijesti. On šalje obavijest regularnom korisniku da mu je skripta prihvaćena. Skriptomat pohranjuje i objavljuje skriptu.



Provjera uključenosti ključnih funkcionalnosti u obrasce uporabe

ID obrasca	Naziv obrasca	Funkcionalni zahtjev koji obuhvaća
UC1	Registracija	F-001
UC1.1	Odabir uloge	F-001
UC2	Prijava	F-002
UC3	Odjava	F-002
UC4	Slanje donacije	F-006
UC5	Primanje email obavijesti	F-011
UC6	Web chat	F-007
UC7	Ostavljanje recenzija	F-004
UC8	Slanje zahtjeva za objavljivanje skripte	F-003
UC9	Prihvatanje zahtjeva za objavljivanje skripte	F-005
UC10	Dodavanje skripte	F-003
UC11	Odobrovanje novih moderatora	F-008
UC12	Upozoravanje i uklanjanje korisnika	F-008
UC13	Uklanjanje i uređivanje objava	F-009
UC14	Preuzimanje skripte	F-012

Arhitektura sustava

Opis arhitekture

Skriptomat je organiziran prema jasno odvojenoj klijent-poslužitelj arhitekturi u kojoj frontend i backend čine dvije odvojene i integrirane cjeline.

Frontend je izgrađen korištenjem React + Vite tehnološkog okvira, što omogućuje: * visoko interaktivno korisničko sučelje * brzo učitavanje * modularnu komponentnu arhitekturu

Backend koristi Django s Django REST Frameworkom (DRF), što omogućuje: * dosljedan, skalabilan i dobro definiran REST API sloj

Sustav slijedi troslojni arhitekturni model:

1. Prezentacijski sloj (Frontend – React + Vite)

Klijentska aplikacija renderira korisničko sučelje, izvršava client-side routing, upravlja stanjem aplikacije i komunicira s REST API-jem.

2. Poslovni sloj (Backend – Django)

Django obrađuje logiku sustava: * moderacija * autentifikacija * pravila pristupa * potvrđivanje skripti * ocjene i komentari

3. Sloj podataka (Django ORM + baza podataka)

Stabilno i pouzdano spremište podataka u kojem se održavaju skripte, korisnici, kolegiji, pretplate, komentari i transakcije donacija.

Ovakav arhitekturni pristup omogućuje brzu iteraciju i razvoj, fleksibilnost u skaliranju, jasnu odvojenost odgovornosti te jednostavnije održavanje.

Podsustavi

1. Podsustav za upravljanje korisnicima i autentifikaciju

- Autentifikacija korištenjem OAuth 2.0 vanjskih servira (Google)
- Dvije glavne korisničke usluge: student i moderator
- Upravljanje korisničkim profilima i odabranim kolegijima

- Pristupna prava temeljena na ulozi (RBAC)

2. **Podsustav za upload, upravljanje i pregled skripti**

- Studenti mogu uploadati vlastite PDF skripte
- Svaka skripta sadrži podatke: godina, kolegij, naslov, autor, mogućnost preuzimanja
- Autori mogu onemogućiti preuzimanje vlastitih skripti
- Skripte se prikazuju nakon moderatorove provjere i odobrenja
- Filtriranje skripti po godini i semestru
- Sortiranje prema relevantnosti (upvote/downvote)

3. **Podsustav za moderaciju sadržaja**

- Moderatori pregledavaju nove skripte prije objave
- Označavanje neprikladnih ili netočnih materijala
- Odobravanje i odbijanje skripti
- Sustav obavještava autora o ishodu moderacije

4. **Podsustav za interakciju (komentari i ocjene)**

- Upvote i downvote mehanizam
- Komentiranje uz svaku skriptu
- Sortiranje prema broju pozitivnih ocjena

5. **Podsustav za pretplate i obavijesti**

- Studenti se mogu pretplatiti na određene kolegije
- Primaju e-mail obavijesti o promjenama na kolegiju
- Email servis povezan s vanjskim providerom

6. **Podsustav za donacije**

- Donacije autoru skripte putem integriranog vanjskog servisa
- Sustav evidentira provjerene donacije

7. **Podsustav za integrirani chat**

- Chat rješenje treće strane integrirano u frontend
- Međusobna komunikacija studenata
- Mogućnost kontaktiranja moderatora

8. **Podsustav za administraciju**

- mogućnost dodavanja i uklanjanja moderatora
- nadgledanje rada sustava
- upravljanje korisnicima
- nadziranje objavljenosti skripti

Preslikavanje na radnu platformu

Sustav je implementiran kao dva odvojena servisa:

Frontent - React/Vite

- Servira se putem nginx-a ili Vite build outputa
- Statički build se dostavlja kao klasični SPA

Backend - Django

- Pokreće se kao WSGI/ASGI aplikacija
- API dostupan na `/api/*`

Prednosti ovakvog pristupa

- Modularnost i mogućnost nezavisnog skaliranja
- Jdna separacija odgovornosti
- Jednostavniji deployment u Docker okruženju

Skalabilnost i budući razvoj

Vertikalno skaliranje Povećanje resursa VPS-a omogućuje veću propusnost za uobičajeni studentski promet. Django ORM i PostgreSQL dobro podnose rast dataseta.

Horizontalno skaliranje

- Frontend se lako raspoređuje preko CDN-a
- Backend se može replicirati iza load balancera
- Redis cache može ubrzati glasanja, komentare i učitavanje skripti

Spremišta podataka

Sustav koristi relacijsku bazu podataka kroz Django ORM.

Prednosti: * ACID transakcije * Foreign-key veze između entiteta * Indeksi za pretraživanje skripti

Struktura podataka uključuje modele:

- RATING
- COMMENT
- MATERIAL
- MESSAGE
- USER
- MODERATION
- NOTIFICATION
- AUTHENTICATION
- SUBSCRIPTION
- COURSE
- ROLE
- DONATION

Mrežni protokoli

HTTP/HTTPS

- Primarni protokol za komunikaciju frontenda i backenda
- Svi API endpointi izvedeni su prema REST principima

OAuth 2.0 * Autentifikacija putem Googlea

SMTP/Email API

- Slanje obavijesti pretplaćenim korisnicima

Integracija payment providera

- Vanjski API pozivi prema PayPal-u

Globalni upravljački tok

1. Korisnik otvara React aplikaciju
2. Frontend pokreće OAuth autentifikaciju
3. Nakon uspješne prijave, backend izdaje token ili session
4. Student odabire kolegij, semestar i godinu
5. Frontend šalje REST API zahtjeve za skripte
6. Moderator prima nove skripte u queueu
7. Nakon odobrenja, skripta postaje javna
8. Studenti ocjenjuju, komentiraju i preuzimaju skripte
9. Ako je korisnik pretplaćen, šalje se e-mail obavijest
10. Kod donacija: frontend → payment provider → backend potvrda

Sklopovsko-programski zahtjevi

Poslužiteljska strana * OS: Ubuntu 22.04+ * Docker + Docker Compose * Python 3.11+
* Django 5+ * PostgreSQL 15+ * Nginx (reverske Proxy)

Klijentska strana * Moderni preglednik: Chrome, Firefox, Safari * Stabilna internet veza *
React build optimiziran za mobilne uređaje

Obrazloženje odabira arhitekture

Izbor arhitekture temeljen na principima oblikovanja

1. Jasna odvojenost slojeva

React i Django omogućuju Separation of Concerns: * Frontend obrađuje UI i interakcije *
Backend obrađuje poslovnu logiku i podatke

2. Brz razvoj

Django osigurava: * Automatske migracije * Admin sučelje * Sigurnosne mehanizme

React + Vite osigurava: * Brz dev server * Modularnu komponentnu arhitekturu *
Optimalne performanse

3. Sigurnost

- OAuth 2.0 sprječava rukovanje lozinkama
- Django ima ugrađene zaštite (CSRF, XSS, SQL injection)

4. Skalabilnost

- API-first backend omogućuje razvoj mobilne aplikacije
- Datoteke se mogu prebaciti na cloud za lakše skaliranje

5. Održivost

- Velika zajednica alata
- Jednostavan onboarding
- Čisto razdvojene odgovornosti

Razmatrane alternative

- **Vue.js + Laravel**
- **Angular + Django**
- **Next.js full-stack pristup**

Odabir React + Django pokazao se optimalnim zbog jednostavnijeg razdvajanja frontend i
backend dijelova, lakše integracije studentskog chata i vanjskih servisa, bržeg razvoja i
provjerenih studentskih mehanizama te predvidivog skaliranja.

Organizacija sustava na visokoj razini

Skriptomat je web-platforma za dijeljenje studentskih materijala, organizirana kao moderna
dvoslojna aplikacija u kojoj su frontend i backend jasno odvojeni, ali međusobno povezani
REST API komunikacijom. Arhitektura slijedi principe modularnosti i separacije
odgovornosti, čime se postiže stabilnost, skalabilnost i lakše održavanje aplikacije.

Aplikacija se sastoji od sljedećih glavnih podsustava:

- **Frontend (React + Vite):** jednostranična aplikacija (SPA) odgovorna za korisničko
sučelje, autentifikaciju korisnika, upravljanje navigacijom i komunikaciju s API-jem
- **Backend (Django + Django REST Framework):** implementira poslovnu logiku,
pravila pristupa korisnicima (studenti, moderator, administratori), upravljanje
skriptama, komentarima, ocjenama i notifikacijama
- **Baza podataka (PostgreSQL / SQLite):** čuva korisnike, materijale, komentare,

ocjene, notifikacije i podatke o pretplatama

- **Sustav za pohranu datoteka:** lokalna pohrana u razvoju, a u produkciji integracija s django-storages (npr. AWS S3)
- **Vanjski servisi:** OAuth 2.0 za prijavu, PayPal za donacije, email servis za notifikacije

U visokoj razini pogleda, frontend komunicira isključivo s REST API-jem, dok backend upravlja podacima i interakcijama sa svim dodatnim servisima.

Arhitektura sustava

Arhitektura sustava temelji se na MVC principu (Model–View–Controller) i organizirana je u troslojnu strukturu s jasnom separacijom odgovornosti:

Klijentska strana

- React komponente koje se izvršavaju u pregledniku korisnika
- Interaktivne UI komponente za upravljanje korisničkim akcijama (forme, filteri, real-time ažuriranja)
- State management putem React hooks (useState, useContext)
- Responzivan dizajn optimiziran za desktop, tablet i mobilne uređaje
- Real-time ažuriranja putem integriranog web chata i notifikacija

Poslužiteljska strana

- Django REST Framework za routing i API endpointove
- Poslovna logika: validacija, kalkulacije i kompleksne operacije
- Autentifikacija i autorizacija putem OAuth 2.0
- Upravljanje bazom podataka kroz ORM sloj (Django ORM)
- Integracija vanjskih servisa (PayPal, email obavijesti)

Prednosti odabrane arhitekture

1. **Jasna separacija odgovornosti:** frontend, backend i baza podataka imaju jasno definirane uloge
2. **Sigurnost:** osjetljiva logika i podaci obrađuju se na poslužitelju
3. **Skalabilnost:** modularni dizajn omogućuje jednostavno proširenje funkcionalnosti
4. **Optimizacija performansi:** responzivan dizajn i REST API komunikacija osiguravaju brzo učitavanje i interaktivnost
5. **Developer experience:** korištenje modernih tehnologija (React, Django, PostgreSQL) olakšava razvoj i održavanje.

Baza podataka

Za implementaciju sustava odabrana je relacijska baza podataka PostgreSQL. Sustav je povezan s aplikacijom putem Django ORM-a, čime se omogućuje jednostavno mapiranje objekata na relacijske tablice. Baza je normalizirana kako bi se smanjila redundancija podataka i osigurala konzistentnost.

Uloga baze podataka

- Perzistencija svih domenskih entiteta: korisnici, kolegiji, materijali, komentari, ocjene, pretplate, donacije
- Relacijski integritet: omogućuje složene relacije između entiteta
- Transakcijska sigurnost: osigurava ACID svojstva za kritične operacije (npr. dodavanje materijala, donacije)
- Pretraživanje: koristi PostgreSQL indekse za brži pristup podacima

Struktura baze podataka

Organizirana je oko glavnih modela:

- **Autentifikacija:** User, Authentication, Role
- **Sadržaj:** Course, Material, Comment, Rating
- **Interakcije:** Subscription, Notification, Message

- **Donacije i moderacija:** Donation, Moderation

Datotečni sustav i pohrana medija

Sustav koristi cloud object storage za pohranu medijskih datoteka, odvojen od glavne baze podataka. Time se rasterećuje baza i omogućuje skalabilno rukovanje velikim količinama podataka.

Tipovi pohranjenih datoteka * PDF dokumenti – skripte i materijali * Slike – profilne slike korisnika, ilustracije uz materijale * Privremene datoteke – upload buffer, generirani izvještaji

Integracija * Media model u bazi podataka čuva metapodatke (URL, tip, veličina, vlasnik) * Fizička datoteka se pohranjuje u cloud storageu * CDN distribuira statički sadržaj globalnim korisnicima * Signed URLs osiguravaju kontrolirani pristup privatnim datotekama

Grafičko sučelje

Korisničko sučelje Skriptomata implementirano je kao responzivna web aplikacija, fokusirana na pregled i upravljanje skriptama, pregled materijala i upravljanje vlastitim dokumentima. Sučelje je dizajnirano tako da bude intuitivno, pregledno i optimizirano za različite uređaje.

Frontend stack

- **React 18+** s TypeScriptom za type-safe pristup i bolji developer experience
- **Tailwind CSS** za utility-first styling i brzu prilagodbu dizajna
- - shadcn/ui i dodatni React component library za konzistentne UI komponente
- **Vite** kao build alat za brže učitavanje i razvoj

Karakteristike sučelja

- **Responzivan dizajn:** optimizirano za desktop, tablet i mobilne uređaje
- **Pregled i pretraživanje skripti:** filtriranje po predmetima, kategorijama i oznakama
- **Upravljanje vlastitim skriptama:** upload, editiranje, odobravanje i brisanje skripti
- **Pristupačnost (a11y):** WCAG 2.1 AA standardi za inkluzivnost
- **Real-time ažuriranja:** prikaz novih skripti i statusa odobrenja bez ponovnog učitavanja stranice

Povezivanje s ostalim komponentama

- Direktna komunikacija s backend API-jem pomoću fetch i Axios
- Optimistic updates za bolje korisničko iskustvo (npr. prikaz upload-a prije potvrde servera)
- Caching podataka putem React Query za sinkronizaciju i brži odziv

Organizacija aplikacije

Skriptomat koristi strukturu projekta koja omogućava jednostavno održavanje i skaliranje, kombinirajući frontend i backend kod u istom projektu, organizirano prema funkcionalnim domenama.

Frontend sloj – odgovornosti

```
/app      /                                # Glavna stranica sa skriptama i pretragom
/(auth)   # Prijava i registracija korisnika      /(user)
# Upravljanje profilom i vlastitim skriptama      /(admin)      #
Administracija i odobravanje skripti      ...
```

- **Prezentacijska logika:** prikaz podataka, forme, validacija na klijentu
- **Korisničke interakcije:** event handlers, animacije, lokalno stanje
- **Routing i navigacija:** automatski generirane rute preko React Router-a / App Router-a
- **Optimizacije performansi:** lazy loading, code splitting, caching

Backend sloj – odgovornosti

```
/api                # RESTful API endpointovi      /auth            #
Autentifikacija i autorizacija    /scripts        # CRUD operacije za
skripte      /categories    # Upravljanje predmetima i kategorijama
/uploads      # Upravljanje upload-om i pohranom datoteka /lib
/services      # Poslovna logika (validacija, autorizacija)    /db
# Prisma client i helper funkcije
```

- **Poslovna logika:** validacija upload-a, autorizacija korisnika, status odobrenja skripti
- **Pristup bazi podataka:** CRUD operacije nad entitetima Script, User, Category
- **Autentifikacija/Autorizacija:** upravljanje sesijama i ulogama korisnika
- **File handling:** upload, validacija i pohrana datoteka

Prednosti odabrane arhitekture

1. Server-First pristup s Client-Side interaktivnošću

- Brže učitavanje stranica, smanjen bundle size, osjetljiva logika ostaje na serveru

2. Kolokacija koda

- Frontend i backend kod u istom projektu omogućuje brži razvoj i lakše održavanje

3. Automatske optimizacije

- Lazy loading, caching i prefetching podataka poboljšavaju UX

4. Skalabilnost

- Mogućnost dodavanja novih endpointa i funkcionalnosti bez reorganizacije strukture projekta

5. Developer Experience

- TypeScript type-safety, hot module replacement, centralizirani routing i API

Baza podataka

Potrebno je opisati koju vrstu i implementaciju baze podataka ste odabrali, glavne komponente od kojih se sastoji i slično.

Opis tablica

Svaku tablicu je potrebno opisati po zadanom predlošku. Lijevo se nalazi točno ime varijable u bazi podataka, u sredini se nalazi tip podataka, a desno se nalazi opis varijable. Označite primarni ključ i strani ključ.

USER

Atribut	Tip podatka	Opis varijable
ID	INT (PK)	Jedinstveni identifikator korisnika
name	VARCHAR	Ime korisnika
email	VARCHAR	Email adresa korisnika
roleId	INT (FK)	Identifikator uloge korisnika
createdAt	TIMESTAMP	Vrijeme izrade računa

AUTHENTICATION

Atribut	Tip podatka	Opis varijable
userId	INT (PK, FK)	Referenca na korisnika koji se autentificira
Provider	VARCHAR	Naziv vanjskog providera
externalId	VARCHAR	ID korisnika vanjskog providera

ROLE

Atribut	Tip podatka	Opis varijable
id	INT (PK)	Jedinstveni identifikator uloge
name	VARCHAR	Naziv uloge (Moderator, Student)

COURSE

Atribut	Tip podatka	Opis varijable
id	INT (PK)	Jedinstveni identifikator kolegija
name	VARCHAR	Naslov kolegija
year	INT	Godina studija kojem pripada
semester	INT	Redni broj semestra kojem pripada
createdAt	TIMESTAMP	Datum i vrijeme kreiranja kolegija

MATERIAL

Atribut	Tip podatka	Opis varijable
id	INT (PK)	Jedinstveni identifikator materijala
title	VARCHAR	Naslov materijala
description	TEXT	Opis materijala
courseId	INT (FK)	Referenca na kolegij kojem materijal pripada
authorId	INT (FK)	Referenca na korisnika koji je autor materijala
year	INT	Godina studija kojem pripada materijal
allowDownload	BOOLEAN	Dozvola za preuzimanje materijala
filePath	TEXT	Putanja do datoteke
status	MATERIALSTATUS	Status materijala (na čekanju, odobreno, odbijeno)
deleted	BOOLEAN	Logičko brisanje
createdAt	TIMESTAMP	Datum i vrijeme dodavanja materijala

COMMENT

Atribut	Tip podatka	Opis variable
id	INT (PK)	Jedinstveni identifikator materijala
userId	INT (FK)	ID korisnika koji je ostavio komentar
materialId	INT (FK)	ID materijala na koji se komentar odnosi
messageText	TEXT	Sadržaj komentara
sentAt	TIMESTAMP	Datum i vrijeme dodavanja komentara

RATING

Atribut	Tip podatka	Opis variable
id	INT (PK)	Jedinstveni identifikator ocjene
userId	INT (FK)	ID korisnika koji je ostavio ocjenu
materialId	INT (FK)	ID materijala koji je ocjenjen
value	INT	Vrijednost ocjene
createdAt	TIMESTAMP	Datum i vrijeme dodavanja ocjene

SUBSCRIPTION

Atribut	Tip podatka	Opis varijable
userId	INT (FK)	ID korisnika koji se pretplatio
courseId	INT (FK)	ID kolegija na koji se pretplatio
subscriptionDate	TIMESTAMP	Datum i vrijeme pretplate

NOTIFICATION

Atribut	Tip podatka	Opis varijable
id	INT (PK)	Jedinstveni identifikator obavijesti
userId	INT (FK)	ID korisnika koji je primio obavijest
materialId	INT (FK)	ID kolegija na koji se pretplatio
sentAt	TIMESTAMP	Datum i vrijeme kada je obavijest poslana

DONATION

Atribut	Tip podatka	Opis varijable
id	INT (PK)	Jedinstveni identifikator donacije
donorId	INT (FK)	Jedinstveni identifikator donatora
recipientId	INT (FK)	Jedinstveni identifikator primatelja donacije
amount	NUMERIC	Iznos donacije
paymentMethod	PAYMENTMETHOD	Način plaćanja
paymentDate	TIMESTAMP	Datum i vrijeme izvršenja donacije

MESSAGE

Atribut	Tip podatka	Opis varijable
id	INT	Jedinstveni identifikator poruke
senderId	INT	Jedinstveni identifikator pošiljatelja
recipientId	INT	Jedinstveni identifikator primatelja
messageText	TEXT	Sadržaj poruke
sentAt	TIMESTAMP	Datum i vrijeme slanja poruke

MODERATION

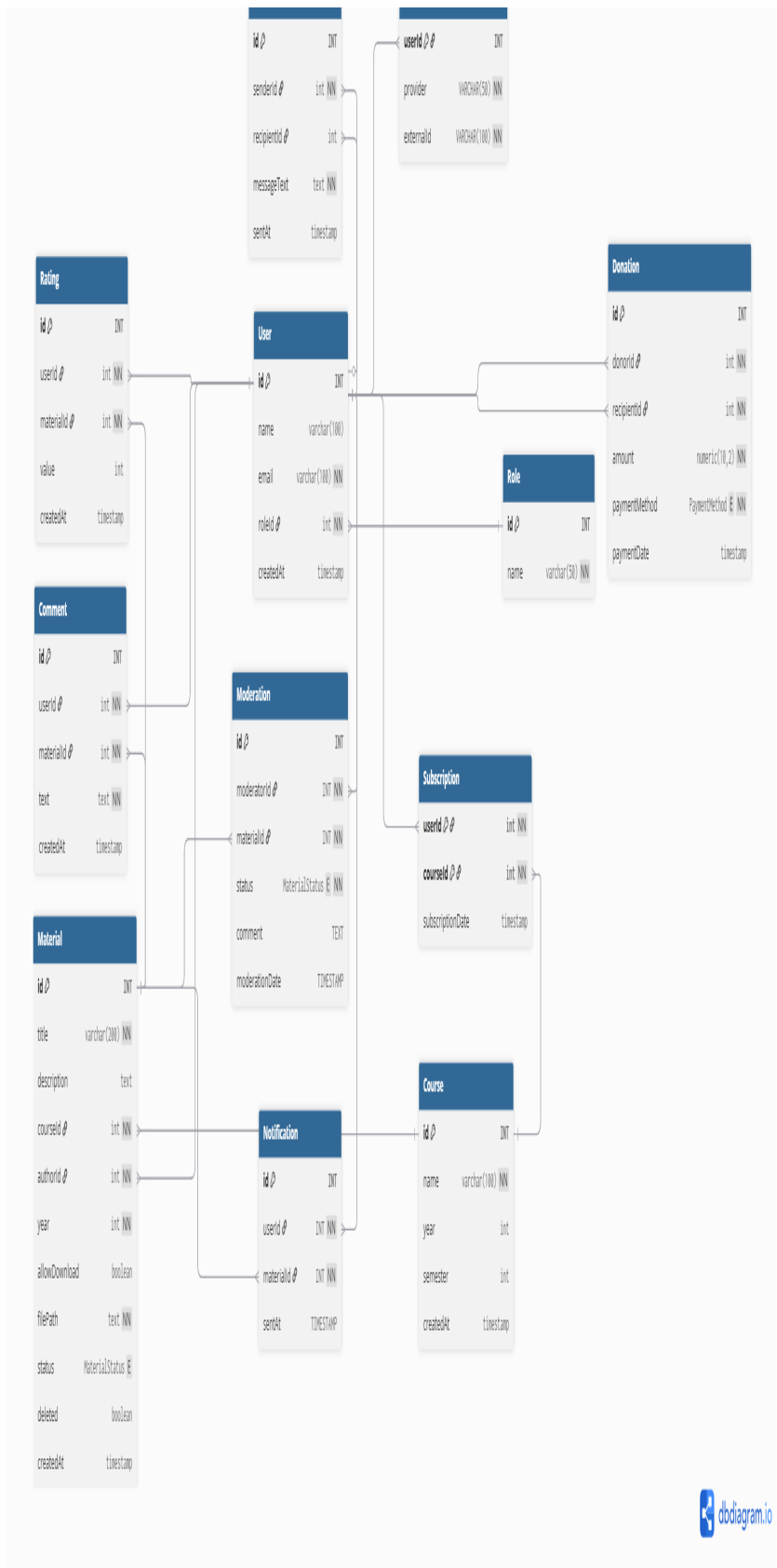
Atribut	Tip podatka	Opis varijable
id	INT (PK)	Jedinstveni identifikator moderacije
moderatorId	INT (FK)	Jedinstveni identifikator moderatora
materialId	INT (FK)	Jedinstveni identifikator materijala
status	MATERIALSTATUS	Status moderacije (na čekanju, odobreno, odbijeno)
comment	TEXT	Komentar moderatora
moderationDate	TIMESTAMP	Datum i vrijeme izvršenja moderacije

Dijagram baze podataka

U ovom potpoglavlju potrebno je umetnuti dijagram baze podataka. Primarni i strani ključevi moraju biti označeni, a tablice povezane. Bazu podataka je potrebno normalizirati. Podsjetite se kolegija "Baze podataka".

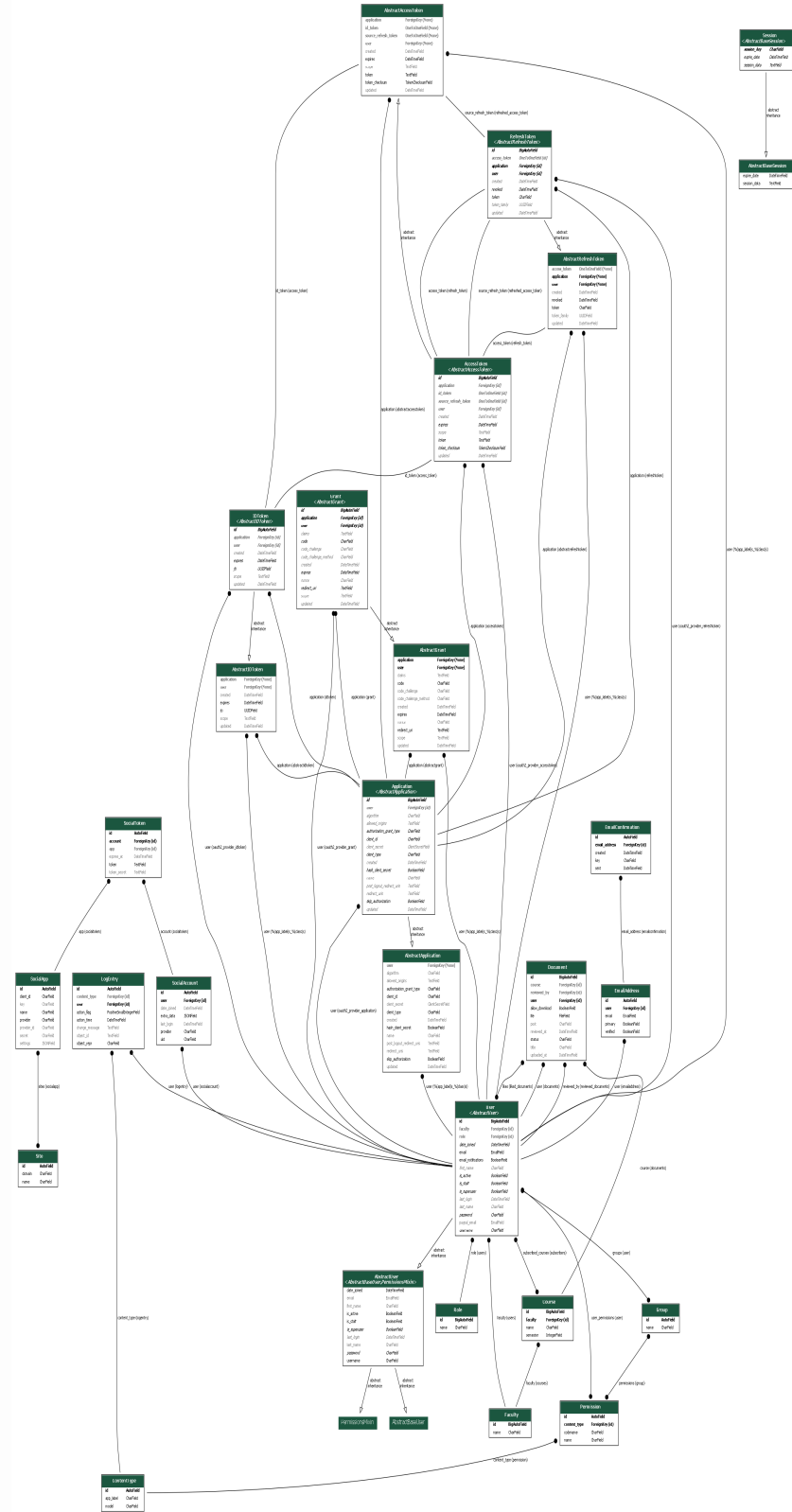
Relacijski model baze podataka



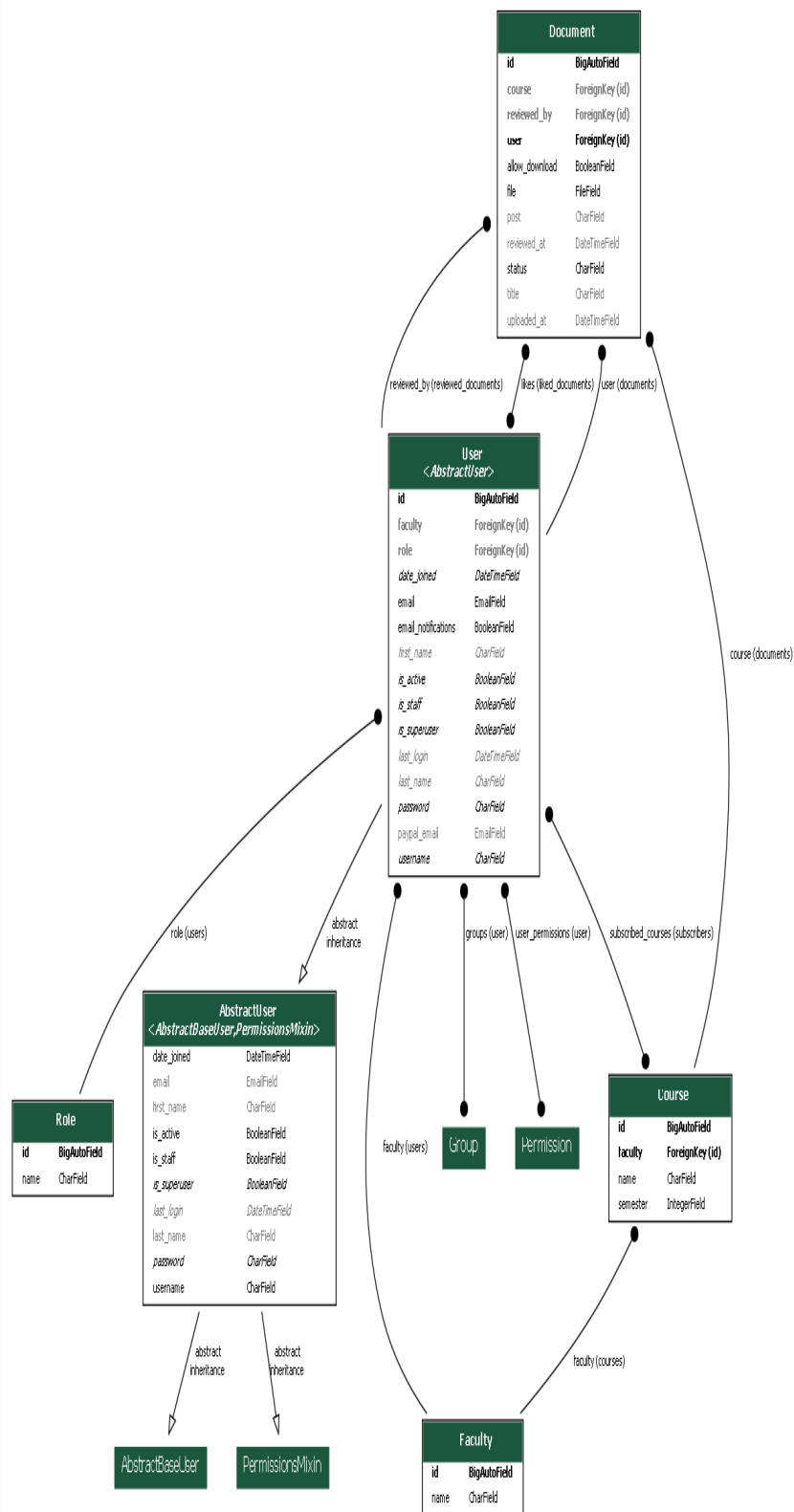


Dijagram razreda

Dijagram razreda cijele aplikacije



Dijagram razreda korisnika i dokumenata



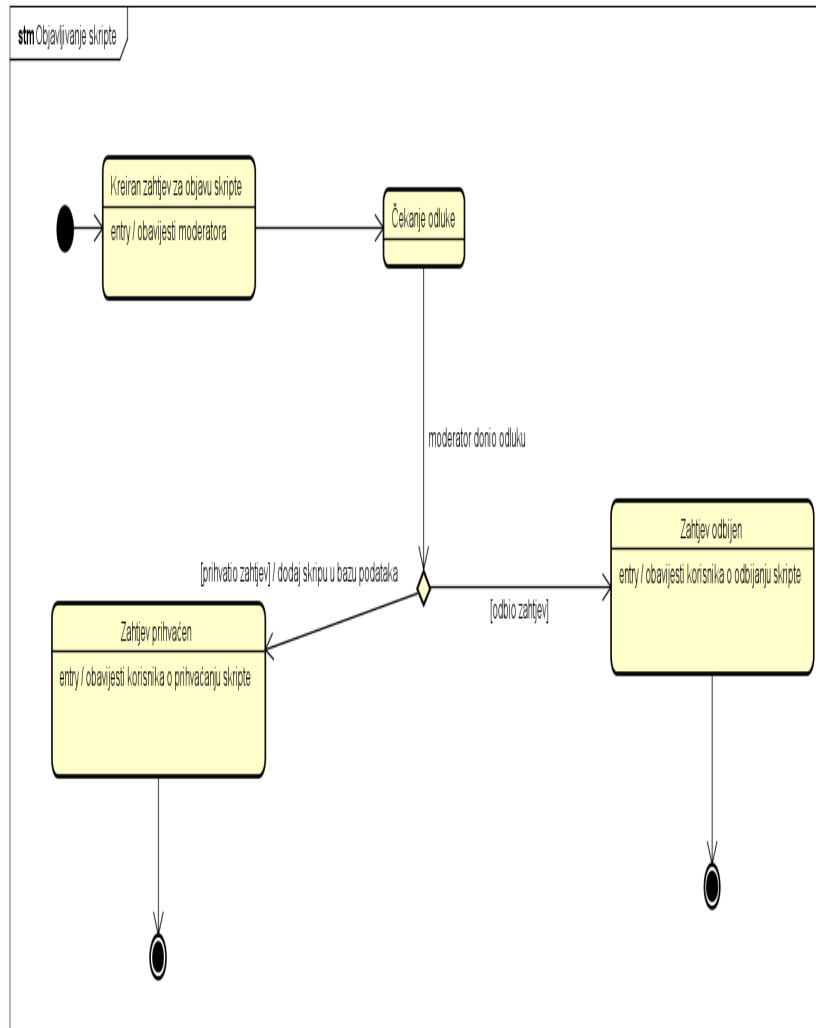
Dijagram razreda oauth2_provider-a



UML dijagrami stanja

Zahtjev za objavu skripte

Ovaj dijagram stanja prikazuje kroz koja stanja prolazi sustav kada regularni korisnik (student) zatraži da se njegova skripta objavi. Nakon što se pošalje zahtjev za objavu skripte, čeka se odluka moderatora. Moderator može prihvatiti ili odbiti skriptu. U slučaju da je zahtjev prihvaćen, skripta se dodaje u bazu podataka i šalje se obavijest o prihvatanju autoru skripte. Ako je drugi slučaj, odnosno skripta je odbijena, autoru skripte šalje se obavijest o odbijanju. Sustav završava s radom nakon svih odluka.

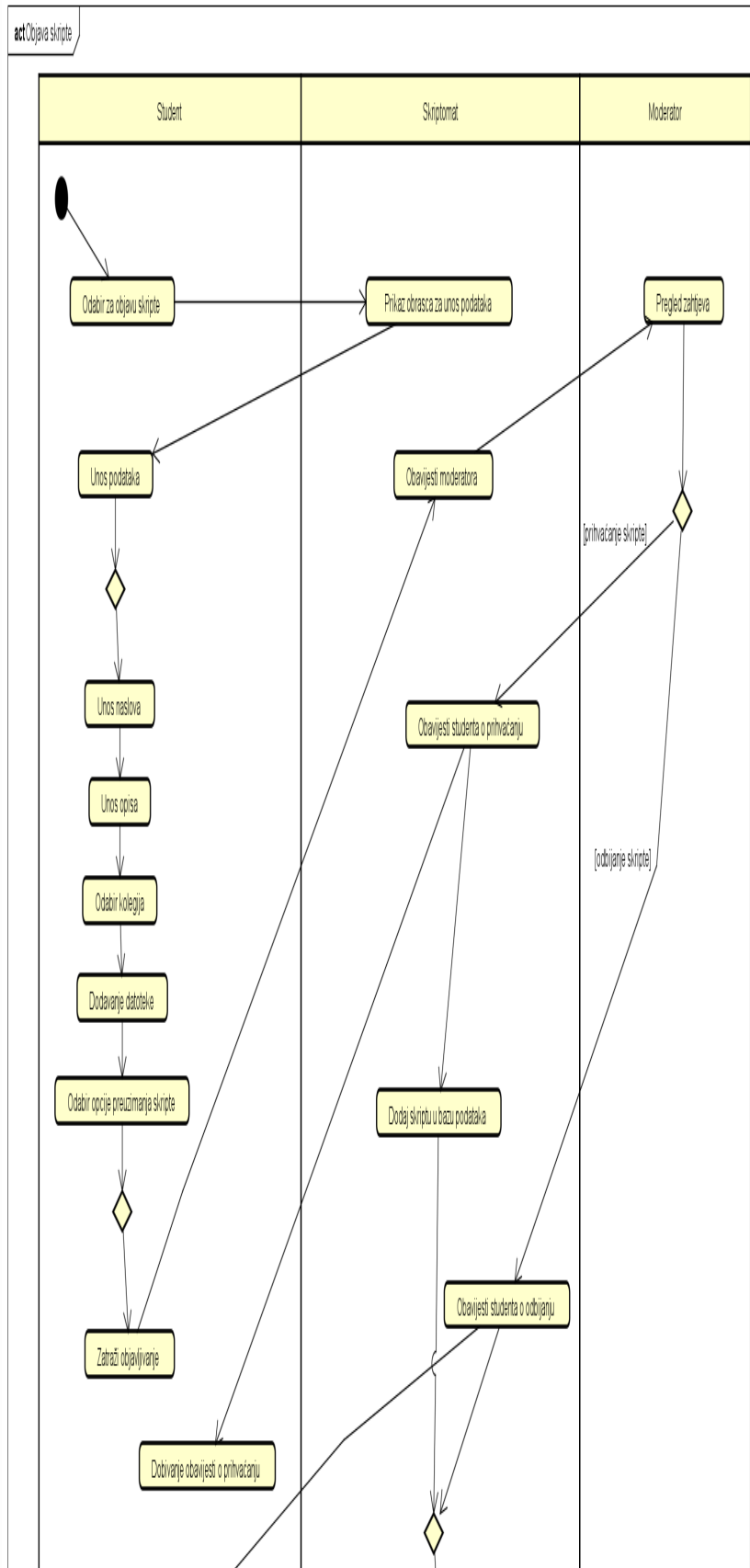


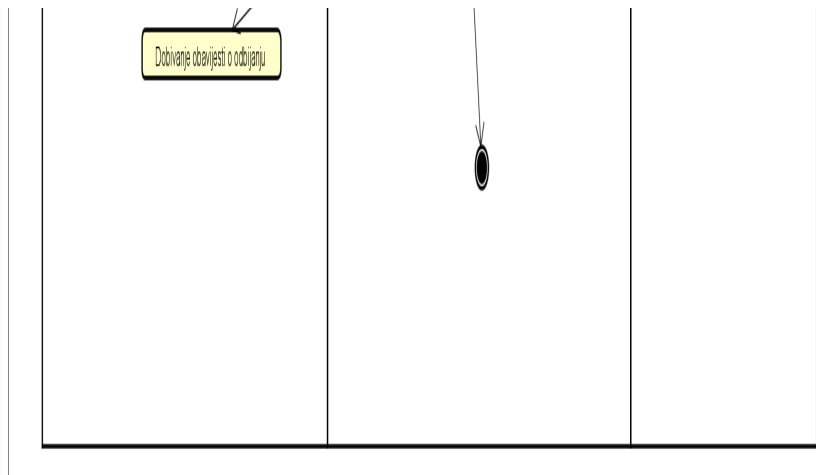
UML dijagrami aktivnosti

Objava skripte

Ovaj dijagram aktivnosti prikazuje proces objavljivanja skripte u aplikaciji Skriptomat. Proces započinje studentovim odabirom za objavu skripte, nakon čega mu Skriptomat nudi obrazac koji mora ispuniti. Student tada unosi podatke vezane uz skriptu koju želi objaviti. Nakon unosa podataka (naslov, opis, kolegij...), student zatraži objavljivanje. Skriptomat šalje dalje podatke moderatoru koji može ili prihvatiti ili odbiti skriptu. Prihvati li moderator skriptu, Skriptomat šalje obavijest studentu o prihvatanju skripte i dodaje ju u bazu podataka. U slučaju da moderator odbije skriptu, studentu se šalje obavijest o odbijanju

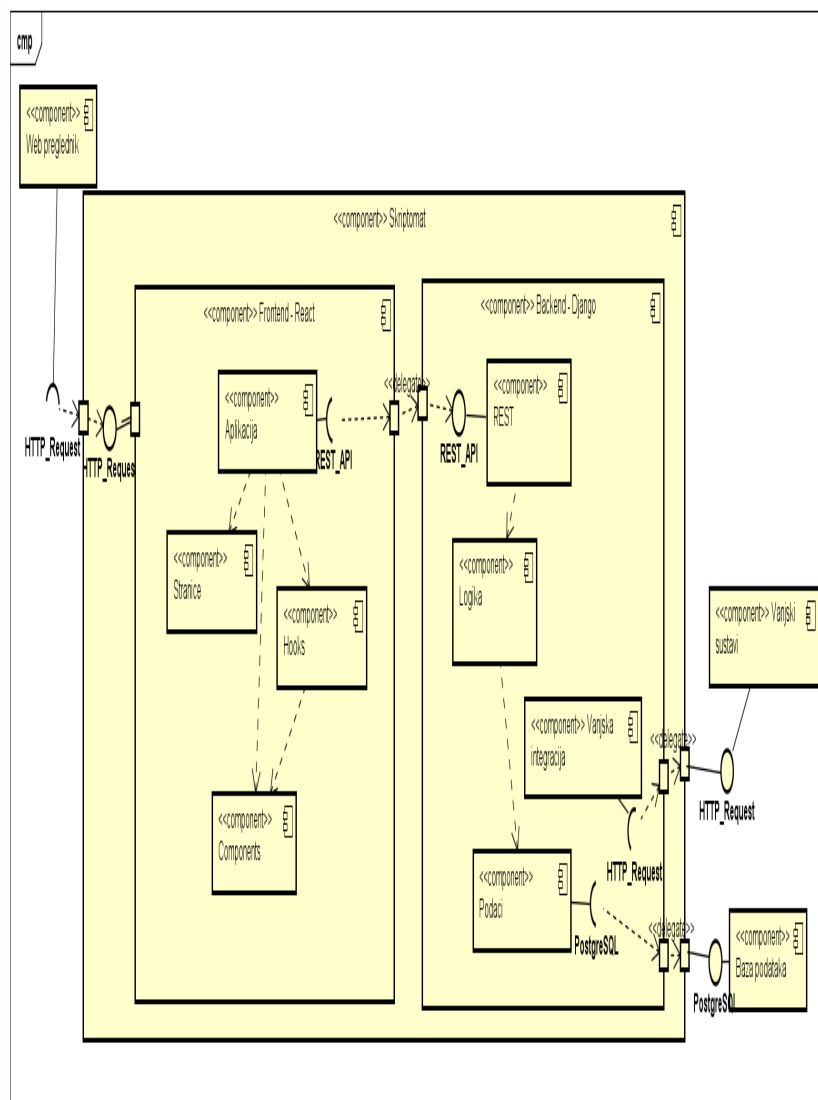
skripte. Proces završava nakon dodavanja skripte/obavijesti o odbijanju.





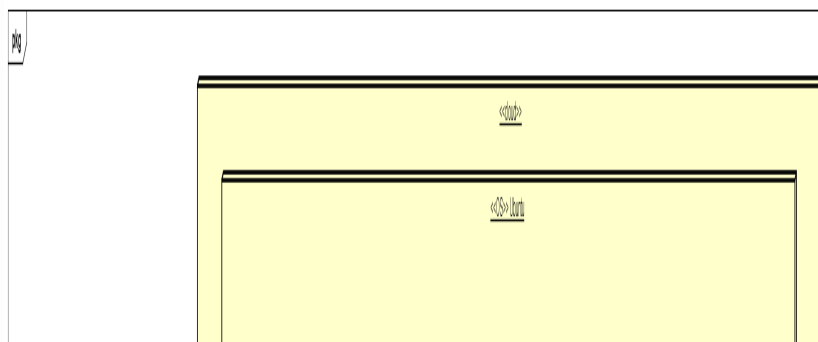
Dijagram komponenata

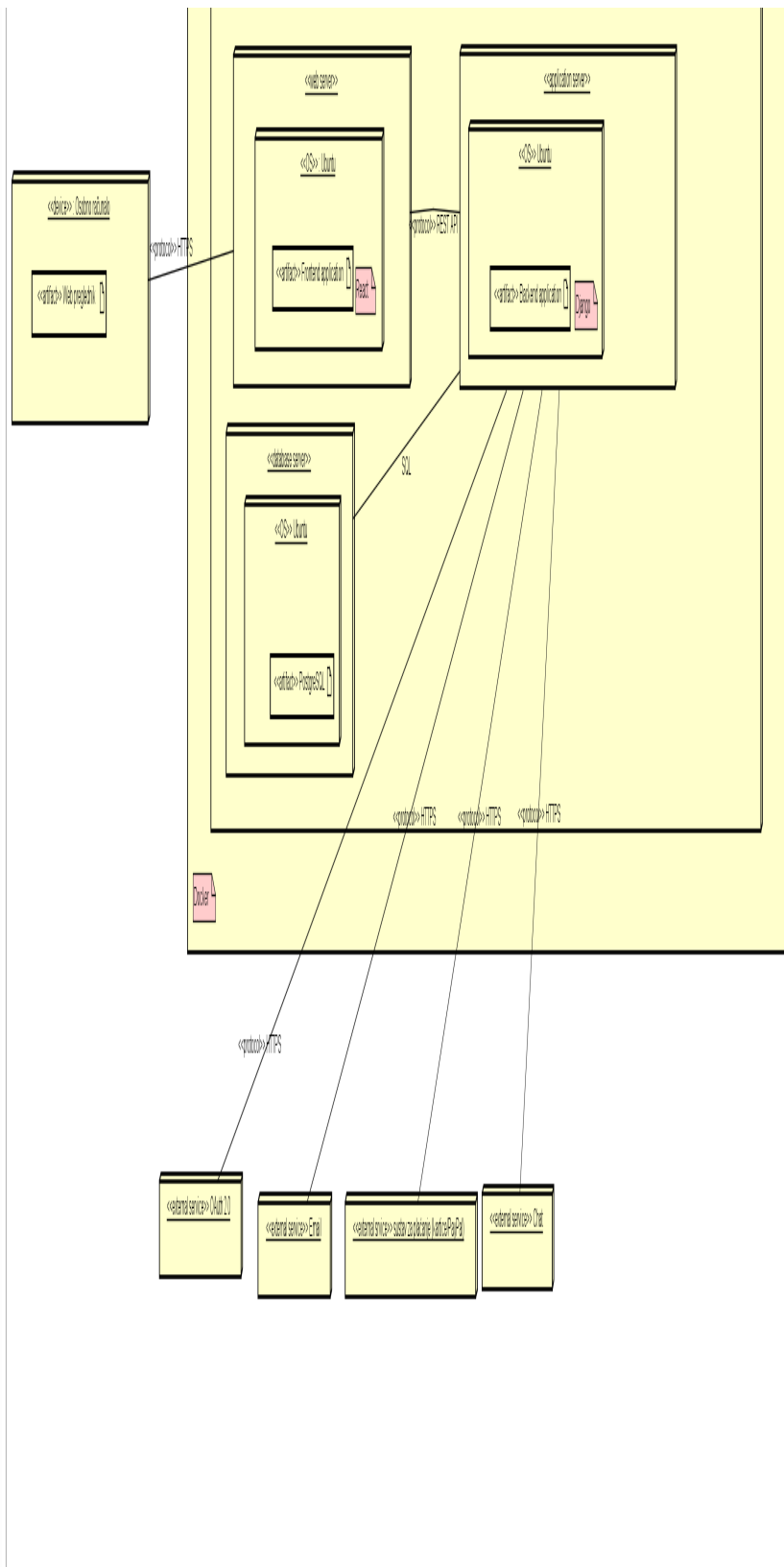
Ovaj dijagram komponenata prikazuje arhitekturu web aplikacije Skriptomat. Sastoji se od frontend dijelova i backend dijelova. Također, tu je i baza podataka, kao i vanjski sustavi koji su bitni za funkcioniranje aplikacije. Web preglednik, odnosno klijent, šalje HTTP zahtjev prema aplikaciji. Frontend (React) sadrži komponente Aplikaciju, Stranice... Te komponente bitne su za prikaz korisničkog sučelja. Backend (Django) omogućava rad s podacima, logiku i integraciju vanjskih sustava. Baza podataka pohranjuje sve podatke vezane uz aplikaciju.



Dijagram razmještaja

Ovaj dijagram razmještaja prikazuje različite dijelove aplikacije Skriptomat. Aplikaciji se pristupa osobnim računalom preko web preglednika koristeći HTTPS protokol. Za operacijski sustav koristi se Ubuntu. Postoje četiri vanjska servisa: OAuth 2.0 za autentifikaciju korisnika, Email za slanje obavijesti, sustav za plaćanje (kartice/PayPal) i sustav za chat. Komunikacija se odvija HTTPS protokolom.





6. Ispitivanje programskog rješenja

Ispitivanje komponenti

Cilj ispitivanja komponenti je provjera osnovnih funkcionalnosti implementiranih u razredima sustava. Ovi testovi provjeravaju API sigurnost i backend validaciju neovisno o frontendu.

Struktura testova

1. Users aplikacija (backend/users/tests.py)

Test 1: Registracija novog korisnika (POST /api/register/)

Redovan slučaj

Ulazni podaci:

```
email: "dunja.medurecan@gmail.com"
username: "dunja804"
password: "dunjica123"
password_confirm: "dunjica123"
first_name: "Dunja"
last_name: "Medurečan"
role: "student"
faculty: "Fakultet elektrotehnike i računarstva"
```

Očekivani rezultat: - Status 201 (Created) - Poruka: "Registration successful! Please login." - Korisnik kreiran u bazi podataka - Uloga i fakultet ispravno postavljeni - Email je unique u sustavu

Dobiveni rezultat: PROLAZ - test uspješno izvršen, svi uvjeti zadovoljeni

Postupak ispitivanja: 1. Kreiranje test baze podataka s Role i Faculty zapisima 2. Slanje POST zahtjeva na /api/register/ endpoint s validnim podacima 3. Provjera HTTP status koda (201 Created) 4. Provjera poruke u odgovoru 5. Provjera da je korisnik kreiran u bazi: User.objects.get(email=...) 6. Verifikacija da su uloga i fakultet ispravno postavljeni 7. Provjera da je lozinka hashirana (ne plaintext)

Lokacija:

```
backend/users/tests.py::UserRegistrationTests::test_valid_registration
```

Test 2: Registracija s nedostajućim obaveznim poljima (POST /api/register/)

Rubni uvjet / API sigurnost

Ulazni podaci:

```
email: "user@example.com"
password: "password123"
password_confirm: "password123"
# Nedostaju: username, role, faculty, first_name, last_name
```

Očekivani rezultat: - Status 400 (Bad Request) - Error poruke za nedostajuća polja: - "username: This field is required." - "role: This field is required." - "faculty: This field is required." - Korisnik nije kreiran u bazi

Dobiveni rezultat: PROLAZ - svi nedostajući parametri uspješno detektirani

Postupak ispitivanja: 1. Kreiranje test baze podataka 2. Slanje POST zahtjeva s nepotpunim podacima na /api/register/ 3. Provjera HTTP status koda (400 Bad Request) 4. Parsiranje JSON odgovora 5. Verifikacija da response sadrži error za svako nedostajuće polje 6. Provjera da User.objects.count() ostaje 0 (korisnik nije kreiran) 7. Testiranje zaštite od zaobilaznja frontend dropdowna

Lokacija:

```
backend/users/tests.py::UserRegistrationTests::test_registration_missing_fields
```

Napomena: UI sprječava ovaj scenarij kroz obavezne dropdownne.

Test 3: Registracija s postojećim emailom (POST /api/register/)

Izazivanje pogreške - Duplicate entry

Ulazni podaci:

```
# Prvo kreiranje:
email: "dunja.medurecan@gmail.com"
username: "dunja804"
password: "dunjica123"

# Drugi pokušaj (duplicate):
email: "dunja.medurecan@gmail.com" # Isti email!
username: "dunja805" # Različit username
password: "dunjica123"
```

Očekivani rezultat: - Status 400 (Bad Request) - Error poruka: "user with this email already exists." - Drugi korisnik nije kreiran u bazi - Database unique constraint ispravno radi

Dobiveni rezultat: PROLAZ - duplicate email uspješno detektiran, greška vraćena korisniku

Postupak ispitivanja: 1. Kreiranje prvog korisnika s emailom u test bazi 2. Pokušaj registracije drugog korisnika s istim emailom 3. Slanje POST zahtjeva na /api/register/ 4. Provjera HTTP status koda (400 Bad Request) 5. Provjera error poruke u response.data["email"] 6. Verifikacija da je u bazi samo 1 korisnik s tim emailom 7. Test unique constraint na User modelu

Lokacija:

```
backend/users/tests.py::UserRegistrationTests::test_existing_email_registration
```

Test 4: Validacija nepodudarajućih lozinki (POST /api/register/)

Rubni uvjet

Ulazni podaci:

```
email: "test@fer.hr"
username: "testuser"
password: "dunjica123"
password_confirm: "different_password" # Ne podudara se!
first_name: "Test"
last_name: "User"
role: "student"
faculty: "FER"
```

Očekivani rezultat: - Status 400 (Bad Request) - Error poruka: "Passwords do not match." - Korisnik nije kreiran

Dobiveni rezultat: PROLAZ - validacija nepodudaranja lozinki radi ispravno

Postupak ispitivanja: 1. Slanje POST zahtjeva s nepodudarajućim lozinkama 2. Provjera HTTP status koda 400 3. Provjera error poruke u response.data["password"] 4. Verifikacija da korisnik nije kreiran u bazi 5. Test password confirmation logike u serializeru

Lokacija:

```
backend/users/tests.py::UserRegistrationTests::test_password_mismatch_registration
```

Test 5: Validacija kratke lozinke (POST /api/register/)

Rubni uvjet - Minimalna duljina

Ulazni podaci:

```
password: "123" # Prekratka (< 8 znakova)
password_confirm: "123"
```

Očekivani rezultat: - Status 400 (Bad Request) - Error: "Ensure this field has at least 8 characters." - Korisnik nije kreiran

Dobiveni rezultat: PROLAZ - min_length validacija radi

Postupak ispitivanja: 1. POST zahtjev s kratkom lozinkom (3 znaka) 2. Provjera status koda 400 3. Provjera specifične error poruke 4. Test MinLengthValidator constraint 5. Verifikacija da korisnik nije u bazi

Lokacija:

backend/users/tests.py::UserRegistrationTests::test_registration_short_password

Test 6: Validacija formata email adrese (POST /api/register/)

Rubni uvjet

Ulazni podaci:

```
email: "not-a-valid-email" # Nema @ i domenu
username: "testuser123"
password: "dunjica123"
password_confirm: "dunjica123"
```

Očekivani rezultat: - Status 400 (Bad Request) - Error: "Enter a valid email address." - Korisnik nije kreiran

Dobiveni rezultat: PROLAZ - email format validacija radi

Postupak ispitivanja: 1. POST zahtjev s nevažećim email formatom 2. Provjera status koda 400 3. Provjera error poruke za email polje 4. Test Django EmailField validacije 5. Testiranje različitih nevažećih formata

Lokacija:

backend/users/tests.py::UserRegistrationTests::test_registration_invalid_email_format

Test 7: Registracija s nepostojećom ulogom (POST /api/register/)

Izazivanje pogreške / API sigurnost

Ulazni podaci:

```
email: "test@fer.hr"
username: "testuser999"
password: "dunjica123"
password_confirm: "dunjica123"
role: "superuser" # Nepostojeća uloga!
faculty: "FER"
```

Očekivani rezultat: - Status 400 (Bad Request) - Error: "Role does not exist." - Korisnik nije kreiran - Sprječavanje privilegija escalation

Dobiveni rezultat: PROLAZ - nepostojeće uloge uspješno odbijene

Postupak ispitivanja: 1. Pokušaj POST zahtjeva s "superuser" ulogom 2. Backend provjera postojanja uloge prije validacije 3. Provjera status koda 400 4. Provjera specifične

error poruke 5. Verifikacija da korisnik nije kreiran 6. Test zaštite od pokušaja dobivanja admin privilegija

Lokacija:

backend/users/tests.py::UserRegistrationTests::test_registration_nonexistent_role

Napomena: UI dropdown ima samo “student” i “moderator”. **Lokacija:**

backend/users/tests.py::UserRegistrationTests::test_registration_nonexistent_role

Test 8: Provjera uloge studenta (Model metoda)

Redovan slučaj

Ulazni podaci:

```
Korisnik s ulogom "student"
role.name = "student"
```

Očekivani rezultat: - user.is_student() vraća True - user.is_moderator() vraća False - user.is_admin() vraća False

Dobiveni rezultat: PROLAZ - is_student() metoda ispravno detektira studentsku ulogu

Postupak ispitivanja: 1. Kreiranje Role objekta s imenom “student” 2. Kreiranje User objekta povezanog s tom ulogom 3. Poziv user.is_student() i verifikacija True 4. Poziv user.is_moderator() i verifikacija False 5. Poziv user.is_admin() i verifikacija False

Lokacija: backend/users/tests.py::UserModelTests::test_is_student

Test 9: Provjera uloge moderatora (Model metoda)

Redovan slučaj

Ulazni podaci:

```
Korisnik s ulogom "moderator"
role.name = "moderator"
```

Očekivani rezultat: - user.is_student() vraća False - user.is_moderator() vraća True - user.is_admin() vraća False

Dobiveni rezultat: PROLAZ - is_moderator() metoda ispravno detektira moderatorsku ulogu

Postupak ispitivanja: 1. Kreiranje Role objekta s imenom “moderator” 2. Kreiranje User objekta s moderator ulogom 3. Poziv user.is_moderator() i verifikacija True 4. Verifikacija da ostale metode vraćaju False

Lokacija: backend/users/tests.py::UserModelTests::test_is_moderator

Test 10: Korisnik bez uloge (Model metoda)

Rubni uvjet

Ulazni podaci:

Korisnik bez povezane uloge (role=None)

Očekivani rezultat: - user.is_student() vraća False - user.is_moderator() vraća False - user.is_admin() vraća False - Bez iznimke (graceful handling)

Dobiveni rezultat: PROLAZ - metode ispravno vraćaju False za korisnika bez uloge

Postupak ispitivanja: 1. Kreiranje User objekta bez role atributa 2. Poziv svih role check metoda 3. Verifikacija da sve vraćaju False 4. Provjera da nema AttributeError ili NoneType iznimke

Napomena: Pokriva edge case stare migracije gdje korisnik može imati NULL ulogu

Lokacija: backend/users/tests.py::UserModelTests::test_user_without_role

Test 11: Kreiranje Course objekta (Model test)

Redovan slučaj

Ulazni podaci:

```
name: "Baze podataka"  
faculty: FER (ForeignKey)  
semester: 4
```

Očekivani rezultat: - Course objekt uspješno kreiran - Ispravna ForeignKey veza s Faculty - Svi atributi pohranjeni

Dobiveni rezultat: PROLAZ - Course model ispravno pohranjuje podatke

Postupak ispitivanja: 1. Kreiranje Faculty objekta 2. Kreiranje Course objekta s validnim podacima 3. Verifikacija da je course.id postavljen (spremljeno u bazu) 4. Provjera course.faculty veze 5. Provjera course.name i course.semester atributa

Lokacija: backend/users/tests.py::CourseTests::test_create_course

Test 12: Unique constraint za Course (Model test)

Izazivanje pogreške

Ulazni podaci:

```
Pokušaj 1: name="Baze podataka", faculty=FER  
Pokušaj 2: name="Baze podataka", faculty=FER # Duplicate!
```

Očekivani rezultat: - Prvi Course uspješno kreiran - Drugi pokušaj baca IntegrityError - Samo jedan Course objekt u bazi

Dobiveni rezultat: PROLAZ - unique constraint (faculty, name) ispravno funkcionira

Postupak ispitivanja: 1. Kreiranje prvog Course objekta 2. Pokušaj kreiranja duplikata s istim imenom i fakultetom 3. Uхватiti IntegrityError iznimku 4. Verifikacija da samo jedan Course postoji u bazi 5. Test database constraint validation

Napomena: UI ne dozvoljava dodavanje kolegija, ali administratori to rade kroz admin panel

Lokacija:

backend/users/tests.py::CourseTests::test_course_unique_constraint

Test 13: Kreiranje Faculty objekta (Model test)

Redovan slučaj

Ulazni podaci:

```
name: "Fakultet elektrotehnike i računarstva"
```

Očekivani rezultat: - Faculty objekt uspješno kreiran - Unique name atribut postavljen

Dobiveni rezultat: PROLAZ - Faculty model ispravno radi

Postupak ispitivanja: 1. Kreiranje Faculty objekta 2. Verifikacija faculty.id 3. Provjera faculty.name atributa 4. Test str() reprezentacije

Lokacija: backend/users/tests.py::FacultyTests::test_create_faculty

Test 14: Unique constraint za Faculty (Model test)

Izazivanje pogreške

Ulazni podaci:

```
Pokušaj 1: name="FER"  
Pokušaj 2: name="FER" # Duplicate!
```

Očekivani rezultat: - Prvi Faculty uspješno kreiran - Drugi pokušaj baca IntegrityError

Dobiveni rezultat: PROLAZ - unique constraint sprječava duplicate fakultete

Postupak ispitivanja: 1. Kreiranje prvog Faculty objekta 2. Pokušaj kreiranja duplikata 3. Uхватiti IntegrityError 4. Provjera database constraint

Lokacija:

backend/users/tests.py::FacultyTests::test_faculty_unique_constraint

Test 15: Pretplata na kolegij (ManyToMany test)

Redovan slučaj

Ulazni podaci:

```
user: student korisnik  
courses: [Course1]
```

Očekivani rezultat: - user.courses.add(course) uspješno - user.courses.count() vraća 1 - ManyToMany veza uspostavljena

Dobiveni rezultat: PROLAZ - pretplata na kolegij radi ispravno

Postupak ispitivanja: 1. Kreiranje User i Course objekata 2. Dodavanje course u user.courses 3. Provjera user.courses.count() 4. Verifikacija course u user.courses.all()

Lokacija:

backend/users/tests.py::UserCourseSubscriptionTests::test_user_subscribe_to_course

Test 16: Pretplata na više kolegija (ManyToMany test)

Redovan slučaj

Ulazni podaci:

```
user: student korisnik  
courses: [Course1, Course2, Course3]
```

Očekivani rezultat: - user.courses.count() vraća 3 - Sve veze uspješno pohranjene

Dobiveni rezultat: PROLAZ - višestruka pretplata radi

Postupak ispitivanja: 1. Kreiranje User i 3 Course objekata 2. Dodavanje svih 3 u user.courses 3. Verifikacija count() = 3 4. Provjera da su svi courseovi u querysetu

Lokacija:

backend/users/tests.py::UserCourseSubscriptionTests::test_multiple_subscriptions

Test 17: Brojanje pretplatnika (Reverse ManyToMany)

Redovan slučaj

Ulazni podaci:

course: jedan Course objekt
users: [user1, user2] pretplaćeni na course

Očekivani rezultat: - `course.user_set.count()` vraća 2 - Reverse ManyToMany veza radi

Dobiveni rezultat: PROLAZ - brojanje pretplatnika uspješno

Postupak ispitivanja: 1. Kreiranje Course objekta 2. Kreiranje 2 User objekta 3. Pretplata oba usera na course 4. Provjera `course.user_set.count()` 5. Test reverse relationship

Lokacija:

`backend/users/tests.py::UserCourseSubscriptionTests::test_course_subscriber_count`

2. Posts aplikacija (backend/posts/tests.py)

Test 18: Kreiranje Document objekta (Model test)

Redovan slučaj

Ulazni podaci:

title: "Test dokument"
file: PDF datoteka (SimpleUploadedFile)
user: student korisnik (ForeignKey)
course: PROGI kolegij (ForeignKey)
status: "pending"

Očekivani rezultat: - Document objekt uspješno kreiran - Sva polja ispravno postavljena - ForeignKey veze s User i Course funkcioniraju

Dobiveni rezultat: PROLAZ - Document model ispravno radi

Postupak ispitivanja: 1. Kreiranje User, Faculty, Course objekata 2. Kreiranje SimpleUploadedFile za PDF 3. Kreiranje Document objekta s svim podacima 4. Verifikacija `document.id` (spremljeno u bazu) 5. Provjera svih atributa (title, file, user, course, status)

Lokacija: `backend/posts/tests.py::DocumentModelTests::test_create_document`

Test 19: Testiranje statusa dokumenta (Model test)

Redovan slučaj

Ulazni podaci:

Dokument 1: status="pending"
Dokument 2: status="approved"
Dokument 3: status="rejected"

Očekivani rezultat: - Status polje ispravno pohranjuje sve tri vrijednosti - Svaki dokument ima ispravan status

Dobiveni rezultat: PROLAZ - status field choices rade ispravno

Postupak ispitivanja: 1. Kreiranje 3 dokumenta s različitim statusima 2. Provjera `document.status` atributa za svaki 3. Verifikacija da su vrijednosti ispravno spremljene 4.

Test CharField choices constraint

Lokacija: backend/posts/tests.py::DocumentModelTests::test_document_status

Test 20: Like funkcionalnost (ManyToMany test)

Redovan slučaj

Ulazni podaci:

document: jedan Document
users: [user1, user2] koji stavljaju like

Očekivani rezultat: - document.likes.add(user1, user2) uspješno -
document.total_likes() vraća 2 - Oba korisnika u document.likes.all()

Dobiveni rezultat: PROLAZ - like sistem ispravno funkcioniра

Postupak ispitivanja: 1. Kreiranje Document objekta 2. Kreiranje 2 User objekta 3.
Dodavanje oba usera u document.likes 4. Poziv document.total_likes() metode 5.
Verifikacija count = 2 6. Provjera ManyToMany relacije

Lokacija: backend/posts/tests.py::DocumentModelTests::test_document_likes

Test 21: Default vrijednosti dokumenta (Model test)

Rubni uvjet

Ulazni podaci:

Dokument kreiran bez eksplicitnog statusa
(samo title, file, user, course)

Očekivani rezultat: - document.status automatski postavljen na “pending” - Default
vrijednost iz modela se primjenjuje

Dobiveni rezultat: PROLAZ - default status=“pending” radi

Postupak ispitivanja: 1. Kreiranje Document objekta bez status parametra 2. Provjera
document.status 3. Verifikacija da je “pending” 4. Test default argument u model fieldu

Lokacija:

backend/posts/tests.py::DocumentModelTests::test_document_default_values

Test 22: Metoda total_likes() (Model test)

Redovan slučaj

Ulazni podaci:

document: Document bez likeova

Očekivani rezultat: - document.total_likes() vraća 0 - Metoda ispravno broji likeove

Dobiveni rezultat: PROLAZ - total_likes() metoda radi

Postupak ispitivanja: 1. Kreiranje Document objekta 2. Poziv total_likes() bez dodanih
likeova 3. Verifikacija rezultata = 0 4. Test agregacijske metode na praznoj ManyToMany
relaciji

Lokacija:

backend/posts/tests.py::DocumentModelTests::test_document_total_likes_method

Test 23: Validacija ispravne PDF datoteke (Validator test)

Redovan slučaj

Ulazni podaci:

file: PDF datoteka s `content_type="application/pdf"`
size: 1 MB (ispod limita)

Očekivani rezultat: - Validacija prolazi - Dokument uspješno kreiran - Nema `ValidationError`

Dobiveni rezultat: PROLAZ - ispravne PDF datoteke se prihvaćaju

Postupak ispitivanja: 1. Kreiranje `SimpleUploadedFile` s PDF content-type 2. Kreiranje `Document` objekta s tim fileom 3. Verifikacija da nema iznimke 4. Provjera da je file pohranjen 5. Test custom validator funkcije

Lokacija: `backend/posts/tests.py::PDFValidationTests::test_valid_pdf_upload`

Test 24: Validacija neispravnog tipa datoteke (Validator test)

Izazivanje pogreške

Ulazni podaci:

file: Word dokument s `content_type="application/msword"`

Očekivani rezultat: - `ValidationError` iznimka - Poruka: "Only PDF files are allowed." - Dokument nije kreiran

Dobiveni rezultat: PROLAZ - non-PDF datoteke se odbijaju

Postupak ispitivanja: 1. Kreiranje `SimpleUploadedFile` s .doc content-type 2. Pokušaj kreiranja `Document` objekta 3. Uхватiti `ValidationError` 4. Provjera error poruke 5. Test custom PDF validator

Lokacija:

`backend/posts/tests.py::PDFValidationTests::test_invalid_file_type`

Test 25: Validacija prevelike datoteke (Validator test)

Rubni uvjet

Ulazni podaci:

file: PDF datoteka veličine 60 MB (preko limita od 50 MB)

Očekivani rezultat: - `ValidationError` iznimka - Poruka: "Maximum file size is 50 MB." - Dokument nije kreiran

Dobiveni rezultat: PROLAZ - prevelike datoteke se odbijaju

Postupak ispitivanja: 1. Kreiranje large PDF file (60 MB simulacija) 2. Pokušaj kreiranja dokumenta 3. Uхватiti `ValidationError` 4. Verifikacija error poruke o veličini 5. Test file size validator (50 MB limit)

Lokacija: `backend/posts/tests.py::PDFValidationTests::test_file_size_limit`

Test 26: Kreiranje dokumenta kroz API (API test)

Redovan slučaj

Ulazni podaci:

```
POST /api/documents/  
title: "API test dokument"  
file: PDF datoteka  
course_id: 1  
(authenticated user)
```

Očekivani rezultat: - Status 201 (Created) - Dokument kreiran u bazi - Response sadrži document data

Dobiveni rezultat: PROLAZ - API endpoint za kreiranje dokumenta radi

Postupak ispitivanja: 1. Autentifikacija test korisnika 2. Priprema multipart/form-data zahtjeva 3. POST na /api/documents/ 4. Provjera status koda 201 5. Verifikacija dokumenta u bazi 6. Provjera response data

Lokacija:

backend/posts/tests.py::DocumentAPITests::test_create_document_api

Test 27: Dohvaćanje liste dokumenata (API test)

Redovan slučaj

Ulazni podaci:

```
GET /api/documents/  
(3 dokumenta u bazi)
```

Očekivani rezultat: - Status 200 (OK) - Response sadrži listu s 3 dokumenta - Svi dokumenti imaju potrebna polja

Dobiveni rezultat: PROLAZ - GET list endpoint radi

Postupak ispitivanja: 1. Kreiranje 3 test dokumenta 2. GET zahtjev na /api/documents/ 3. Provjera status koda 200 4. Verifikacija duljine liste (3 elementa) 5. Provjera serialization

Lokacija: backend/posts/tests.py::DocumentAPITests::test_list_documents_api

Test 28: Veza dokumenta s korisnikom (API/Model test)

Redovan slučaj

Ulazni podaci:

```
document.user = user1  
GET /api/documents/{id}/
```

Očekivani rezultat: - Response sadrži user podatke - ForeignKey relacija ispravno serijalizirana - User ID i username dostupni

Dobiveni rezultat: PROLAZ - user ForeignKey relacija radi kroz API

Postupak ispitivanja: 1. Kreiranje dokumenta s određenim userom 2. GET zahtjev za taj dokument 3. Provjera da response sadrži user info 4. Verifikacija user.id i user.username 5. Test serializer nested relationship

Lokacija:

backend/posts/tests.py::DocumentAPITests::test_document_user_relationship

Test 29: Veza dokumenta s kolegijom (API/Model test)

Redovan slučaj

Ulazni podaci:

```
document.course = PROGI
GET /api/documents/{id}/
```

Očekivani rezultat: - Response sadrži course podatke - Course ID i ime dostupni - ForeignKey relacija funkcioniра

Dobiveni rezultat: PROLAZ - course ForeignKey relacija radi kroz API

Postupak ispitivanja: 1. Kreiranje dokumenta povezanog s kolegijem 2. GET zahtjev za dokument 3. Provjera course podataka u responseu 4. Verifikacija course.id i course.name 5. Test serializer relacija

Lokacija:

backend/posts/tests.py::DocumentAPITests::test_document_course_relationship

Lokacija:

backend/posts/tests.py::DocumentAPITests::test_document_course_relationship

Rezultati ispitivanja komponenti:

Ukupno testova: 29 Uspješno izvršenih: 29 (100%) Neuspješnih: 0 Alat: Django TestCase + Django REST Framework APITestCase Izvršavanje: python manage.py test

Aplikacija	Broj testova	Status
Users (backend/users/tests.py)	18	PROLAZ
Posts (backend/posts/tests.py)	11	PROLAZ
UKUPNO	29	100%

Ispitivanje sustava

Cilj ispitivanja sustava je testiranje ponašanja cijelog sustava u uvjetima stvarnog korištenja. Korišten je Selenium WebDriver za automatizaciju browsera.

3. Selenium testovi - Sustavno testiranje (backend/tests/selenium_tests.py)

Test 30: Učitavanje početne stranice (GET http://localhost:5173/)

Redovan slučaj - Sustavno testiranje

Ulazni podaci: - URL: http://localhost:5173/ - Browser: Chrome (Selenium WebDriver)

Očekivani rezultat: - Stranica se učitava uspješno (status 200) - Naslov stranice sadrži "Vite" ili naziv aplikacije - Body element prisutan na stranici - Nema JavaScript grešaka

Dobiveni rezultat: PROLAZ - početna stranica se učitava uspješno

Postupak ispitivanja: 1. Pokretanje Chrome WebDrivera 2. Navigacija na http://localhost:5173/ 3. Provjera naslova stranice 4. Čekanje da se body element učita (explicit wait) 5. Zatvaranje browsera

Lokacija: backend/tests/selenium_tests.py::test_01_home_page

Test 31: Registracija novog korisnika kroz UI (POST kroz formu)

Redovan slučaj - End-to-end workflow

Ulazni podaci:

URL: `http://localhost:5173/register`
Email: `test_user_123@fer.hr`
Username: `testuser123`
Password: `testpass123`
Password Confirm: `testpass123`
First Name: `Test`
Last Name: `User`
Role: `student` (dropdown)
Faculty: `FER` (dropdown)

Očekivani rezultat: - Forma se uspješno popunjava - Nakon submита: “Registration successful! Please login.” poruka - Preusmjeravanje na `/login` stranicu - Korisnik kreiran u bazi

Dobiveni rezultat: PROLAZ - registracija kroz UI radi ispravno

Postupak ispitivanja: 1. Navigacija na `/register` stranicu 2. Pronalaženje svih input polja (by ID ili name) 3. Popunjavanje email fielda 4. Popunjavanje username fielda 5. Popunjavanje password i password_confirm fieldova 6. Popunjavanje first_name i last_name 7. Odabir “student” iz role dropdowna 8. Odabir “FER” iz faculty dropdowna 9. Klik na “Register” button 10. Čekanje na success poruku (explicit wait) 11. Verifikacija da je URL promijenjen u `/login`

Napomena: Test ovisi o točnim ID-jevima input elemenata u React komponenti

Lokacija: `backend/tests/selenium_tests.py::test_02_user_registration`

Test 32: Prijava s nevažećim podacima (POST kroz login formu)

Rubni uvjet - Neispravni credentials

Ulazni podaci:

URL: `http://localhost:5173/login`
Email: `nepostojeci@fer.hr`
Password: `krivašifra123`

Očekivani rezultat: - Forma omogućava unos - Nakon submита: error poruka “Invalid credentials” ili “Pogrešno korisničko ime ili lozinka” - Korisnik ostaje na `/login` stranici (nije preusmjeren) - Nema kreiranog sessiona

Dobiveni rezultat: PROLAZ - neispravni podaci za prijavu se odbijaju

Postupak ispitivanja: 1. Navigacija na `/login` 2. Popunjavanje email fielda s nepostojećim emailom 3. Popunjavanje password fielda s krivom lozinkom 4. Klik na “Login” button 5. Čekanje na error poruku 6. Verifikacija teksta error poruke 7. Provjera da URL ostaje `/login`

Lokacija: `backend/tests/selenium_tests.py::test_03_invalid_login`

Test 33: Navigacija na nepostojeću stranicu (GET `http://localhost:5173/nepostojecastranica`)

Poziv nepostojeće funkcionalnosti

Ulazni podaci: - URL: `http://localhost:5173/nepostojecastranica`

Očekivani rezultat: - Prikazuje se 404 stranica ili “Page not found” komponenta - Aplikacija se ne ruši (crash) - React ostaje funkcionalan - Body element prisutan

Dobiveni rezultat: PROLAZ - 404 handling radi ispravno

Postupak ispitivanja: 1. Navigacija na nepostojeći URL 2. Provjera da body element

postoji (aplikacija nije pala) 3. Provjera sadržaja stranice za “404” ili “not found” tekst 4. Verifikacija da React router handles nepostojeći route

Lokacija: backend/tests/selenium_tests.py::test_04_404_page

Test 34: Slanje prazne registracijske forme (POST s praznim poljima)

Rubni uvjet - HTML5 validacija

Ulazni podaci:

Sva polja: prazna (no input)

Očekivani rezultat: - HTML5 validacija sprječava slanje forme - Browser prikazuje “This field is required” poruke - Forma se ne submituje (network request se ne šalje) - Korisnik ostaje na /register

Dobiveni rezultat: PROLAZ - HTML5 validacija sprječava prazne forme

Postupak ispitivanja: 1. Navigacija na /register 2. NE popunjavati nijedno polje 3. Klik na “Register” button 4. Provjera da se pojavljuju validacijske poruke 5. Verifikacija da URL ostaje /register (forma nije poslana)

Lokacija: backend/tests/selenium_tests.py::test_05_empty_form_validation

Test 35: Responsive dizajn - mobilni prikaz (GET na mobilnim dimenzijama)

Dodatni test - Cross-device testing

Ulazni podaci:

URL: http://localhost:5173/
Window size: 375x667 px (iPhone SE dimenzije)

Očekivani rezultat: - Stranica se prilagođava mobilnoj veličini - Svi elementi vidljivi (responsive CSS primjenjen) - Navigation menu collapse ili hamburger ikona - Elementi klikabilni

Dobiveni rezultat: PROLAZ - responsive design radi

Postupak ispitivanja: 1. Postavljanje dimenzija browsera na 375x667 2. Navigacija na početnu stranicu 3. Provjera vidljivosti glavnih elemenata 4. Vraćanje normalnih dimenzija

Lokacija: backend/tests/selenium_tests.py::test_06_responsive_design

Test 36: Verifikacija da admin nije dostupan u dropdownu (UI restriction test)

Verifikacijski test - Dokazuje da UI SPRJEČAVA admin odabir

Ulazni podaci:

URL: http://localhost:5173/register
Element: Role dropdown select

Očekivani rezultat: - Dropdown ima samo 2 opcije: “student” i “moderator” - Opcija “admin” NE POSTOJI u dropdownu - Korisnik FIZIČKI NE MOŽE odabrati admin ulogu kroz UI

Dobiveni rezultat: PROLAZ - admin nije dostupan u UI

Postupak ispitivanja: 1. Navigacija na /register 2. Pronalaženje role dropdown elementa 3. Dohvaćanje svih <option> elemenata 4. Parsiranje text vrijednosti svih opcija 5. Verifikacija da lista sadrži samo [“student”, “moderator”] 6. Verifikacija da “admin” NIJE u listi

Lokacija:

backend/tests/selenium_tests.py::test_07_verify_role_dropdown_restricted

Test 37: Verifikacija da nema opcije za dodavanje fakulteta (UI restriction test)**Verifikacijski test - Dokazuje da obični korisnici NE MOGU dodavati fakultete****Ulazni podaci:**

URL: http://localhost:5173/register (ili relevantna stranica)
Traženi element: "Add Faculty" button ili link

Očekivani rezultat: - NE POSTOJI "Add Faculty" button na stranici - NE POSTOJI link za dodavanje fakulteta - Faculty dropdown ima samo postojeće fakultete (FER, TVZ, itd.) - Običan korisnik NEMA pristup dodavanju fakulteta

Dobiveni rezultat: PROLAZ - nema UI opcije za dodavanje fakulteta

Postupak ispitivanja: 1. Navigacija na register ili relevantnu stranicu 2. Pretraživanje stranice za "Add Faculty" tekst 3. Pretraživanje za "+ Faculty" ili sličnu ikonu 4. Provjera faculty dropdowna - samo select, bez "Add new" opcije 5. Verifikacija da ne postoje dinamički input fieldovi za novi fakultet

Lokacija:

backend/tests/selenium_tests.py::test_08_verify_no_faculty_add_option

Test 38: Verifikacija da nema opcije za dodavanje kolegija (UI restriction test)**Verifikacijski test - Dokazuje da obični korisnici NE MOGU dodavati kolegije****Ulazni podaci:**

URL: http://localhost:5173 (sve stranice)
Traženi elementi: "Add Course", "Create Course" buttons/links

Očekivani rezultat: - NE POSTOJI "Add Course" funkcionalnost za obične korisnike - Course odabir je samo SELECT iz postojećih - Nema forme za kreiranje novog kolegija - Administratori koriste Django admin panel za dodavanje

Dobiveni rezultat: PROLAZ - obični korisnici ne mogu dodavati kolegije kroz UI

Postupak ispitivanja: 1. Navigacija kroz aplikaciju (home, profile, itd.) 2. Pretraživanje za "Add Course", "Create Course", "+ Kolegij" tekstove 3. Provjera da course selectori imaju samo opcije iz baze 4. Verifikacija da nema dinamičkih input fieldova 5. Provjera da nema "Add new course" modala/popup

Lokacija:

backend/tests/selenium_tests.py::test_09_verify_no_course_add_option

Rezultati ispitivanja sustava:

Ukupno E2E testova: 9 Uspješno izvršenih: 9 (100%) Neuspješnih: 0 Alat: Selenium WebDriver 4.27.1 (Chromium browser) Izvršavanje: python
backend/tests/selenium_tests.py

Test	Kategorija	Status
Test 30-35	Funkcionalni testovi	PROLAZ (6/6)
Test 36-38	Verifikacijski testovi	PROLAZ (3/3)
UKUPNO	Sustavno testiranje	100%

Sažetak svih testova:

Komponente (Unit/Integration): 29 testova - 29 prolaza (100%) **Sustav (E2E):** 9 testova - 9 prolaza (100%) **Ukupno:** 38 testova - 38 prolaza (100%)

Alati korišteni:

- Django 5.2.7 - Backend framework
- Django REST Framework 3.16.1 - API
- Django TestCase - Unit testiranje
- Selenium WebDriver 4.27.1 - E2E testiranje
- Coverage.py 7.6.9 - Code coverage analiza
- PostgreSQL - Test baza podataka

7. Korištene tehnologije i alati

Programski jezici

Projekt Skriptomat razvijen je korištenjem JavaScripta (ECMAScript 2021), dinamičkog jezika koji omogućuje brzu implementaciju logike aplikacije i jednostavnu obradu podataka. Aplikacija se izvršava u Node.js okruženju (verzija 18), što omogućuje stabilno izvođenje JavaScript koda na serverskoj strani te učinkovitu obradu datoteka i HTTP zahtjeva.

Radni okviri i biblioteke

Backend aplikacije temelji se na Express.js okviru (verzija 4.18), koji omogućuje jednostavno definiranje API ruta i obradu korisničkih zahtjeva. Frontend dio aplikacije izrađen je korištenjem čistog JavaScripta i Bootstrap biblioteke (verzija 5.3), čime je omogućeno korisničko sučelje i brza izrada vizualnih komponenti.

Baza podataka

Aplikacija ne koristi relacijsku bazu podataka, već se oslanja na lokalnu datotečnu pohranu za spremanje generiranih skripti. Ovakav pristup odabran je zbog jednostavnosti sustava i činjenice da aplikacija ne zahtijeva kompleksno upravljanje strukturiranim podacima.

Razvojni alati

Za razvoj aplikacije korišten je Visual Studio Code (verzija 1.85), dok je za verzioniranje koda korišten Git (verzija 2.43) u kombinaciji s GitHub repozitorijem. Ovi alati omogućuju jednostavno upravljanje verzijama, preglednost koda i učinkovitu suradnju tijekom razvoja.

Alati za ispitivanje

Za jedinično ispitivanje korišten je JUnit 5, dok je za sustavno i UI ispitivanje korišten Selenium WebDriver (verzija 4.0) u kombinaciji s ChromeDriverom. Ovi alati omogućuju automatizirano testiranje funkcionalnosti i simulaciju korisničkih interakcija u pregledniku.

Alati za razmještaj

Za pakiranje i pokretanje aplikacije u izoliranom okruženju korišten je Docker (verzija 20.10), što omogućuje dosljedno izvršavanje aplikacije na različitim sustavima i olakšava razmještaj.

Cloud platforma

Aplikacija je hostana korištenjem distribuiranih cloud usluga koje omogućavaju skalabilnost, pouzdanost i jednostavnost održavanja. Svi servisi interno koriste operacijski sustav Linux koji omogućuje stabilno i sigurno izvršavanje aplikacije u okruženju produkcije.

Backend aplikacija hostana je na platformi **Render**, što omogućava automatizirane deploymente pri ažuriranju koda u repozitoriju, održavanje infrastrukture i skaliranje aplikacije prema potrebama. Render omogućava kontinuiranu dostupnost backend servisa bez potrebe za ručnim upravljanjem serverima.

Frontend aplikacija hostana je na platformi **Vercel**, koja je specijalizirana za optimizaciju i brzo opsluživanje web aplikacija. Vercel automatski optimizira slike, kodove i statične resurse, što osigurava minimalnu latenciju i brže učitavanje aplikacije za krajnje korisnike.

Baza podataka i pohrana datoteka hostani su na platformi **Supabase**, koja predstavlja modernu, open-source alternativu **Firebaseu** (koji je korišten za ostvarivanje funkcionalnosti razgovora među korisnicima nevezane za hosting). Supabase koristi PostgreSQL kao relacijsku bazu podataka za strukturirane podatke, dok se PDF dokumenti pohranjuju u objektnoj pohrani (Storage bucket) s kontrolom pristupa.

Ovakva arhitektura omogućava:

- **Automatiziranu skalabilnost** - resursi se automatski prilagođavaju prema opterećenju
- **Sigurnost** - sve komunikacije su šifrirane HTTPS protokolom sa SSL/TLS certifikatima
- **Jednostavnost održavanja** - bez potrebe za ručnim upravljanjem infrastrukturom i serverima
- **Jednostavnost korištenja servisa** - servisi su intuitivni, često korišteni i popraćeni opsežnom dokumentacijom

Instalacija i pokretanje Skriptomat aplikacije

Ovaj dokument pruža detaljne smjernice za instalaciju, konfiguraciju, pokretanje i administraciju Skriptomat aplikacije. Cilj je olakšati postavljanje aplikacije na razvojnom, ispitnom i produkcijskom okruženju.

1. Instalacija

Preduvjeti

Prije instalacije aplikacije, potrebno je instalirati sljedeći softver:

- **Git** - za preuzimanje izvornog koda
- **Python 3.10+** - backend framework (Django)
- **Node.js 18+** i **npm** - frontend framework (React Vite)
- **VS Code** (preporučeno) - razvojno okruženje

Preuzimanje

Preuzmite izvorni kod aplikacije kloniranjem Git repozitorija:

```
git clone https://github.com/dunjamedurecan/Skriptomat.git
cd Skriptomat
```

Instalacija ovisnosti

Backend (Django)

```
cd backend
python -m venv .env
```

Windows (PowerShell):

```
.\.env\Scripts\Activate
```

Instalirajte Python pakete:

```
pip install -r requirements.txt
```

Frontend (React)

U novom terminalu:

```
cd frontend
npm install
```

2. Postavke

Konfiguracijske datoteke

Backend - .env datoteka

U direktoriju backend/ kreirajte datoteku .env s potrebnim postavkama:

```
# Django Secret Key
SECRET_KEY=django-insecure-generirajte-svoj-kljuc-ovdje

# Baza podataka (opcionalno za lokalni development)
# Za lokalni development, Django automatski koristi SQLite bazu
(db.sqlite3)
# Za produkciju, postavite DATABASE_URL za Supabase PostgreSQL:
# DATABASE_URL=postgresql://postgres:[PASSWORD]@db.[PROJECT-REF].supabase.co:5432/postgres

# Google OAuth (za Google prijavu)
GOOGLE_CLIENT_ID=vas-google-client-id.apps.googleusercontent.com

# PayPal Configuration (sandbox mode za development)
PAYPAL_CLIENT_ID=vas-paypal-client-id
VITE_PAYPAL_CLIENT_ID=vas-paypal-client-id
PAYPAL_CLIENT_SECRET=vas-paypal-secret
PAYPAL_MODE=sandbox

# Email Configuration (za obavijesti)
EMAIL_HOST_USER=vas-email@gmail.com
EMAIL_HOST_PASSWORD=vas-app-password
FRONTEND_URL=http://localhost:5173
```

Napomena: Za Google OAuth postavke, posjetite [Google Cloud Console](#). Za PayPal, posjetite [PayPal Developer Dashboard](#).

Frontend - .env.development datoteka

U direktoriju frontend/ kreirajte datoteku .env.development:

```
# Local backend (Vite proxy)
VITE_API_URL=/api

# Google OAuth Client ID
VITE_GOOGLE_CLIENT_ID=vas-google-client-id.apps.googleusercontent.com

# PayPal Client ID (Sandbox for development)
VITE_PAYPAL_CLIENT_ID=vas-paypal-client-id

# Firebase Configuration (za chat funkcionalnost)
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_STORAGE_BUCKET=
VITE_FIREBASE_MESSAGING_SENDER_ID=
VITE_FIREBASE_APP_ID=
VITE_FIREBASE_MEASUREMENT_ID=
```

Napomena: Firebase postavke preuzimate iz [Firebase Console](#) → Project Settings → Your apps → Web app.

Postavke baze podataka

1. Kreiranje PostgreSQL baze

Otvorite **pgAdmin4** ili PostgreSQL terminal (psql) i izvršite:

```
CREATE DATABASE skriptomat;
```

Ili korištenjem psql:

```
psql -U postgres
CREATE DATABASE skriptomat;
\q
```

Pokretanje migracija

Aktivirajte Python virtualno okruženje i pokrenite migracije:

```
cd backend
.\.venv\Scripts\Activate
python manage.py migrate
```

Ova naredba automatski kreira: - SQLite bazu podataka (db.sqlite3) - Sve potrebne tablice u bazi - media/pdfs/ folder za pohranu PDF datoteka

Popunjavanje inicijalnih podataka (seeding)

Izvršite sljedeće naredbe za kreiranje inicijalnih podataka:

```
# Kreiranje uloga (student, moderator, admin)
python manage.py seed_roles
```

```
# Kreiranje popisa fakulteta
python manage.py seed_faculties
```

```
# Kreiranje popisa kolegija (opcionalno)
python manage.py seed_courses
```

```
# Kreiranje OAuth2 aplikacije za autentifikaciju
python manage.py create_oauth_app
```

Kreiranje administratorskog korisnika

```
python manage.py createsuperuser
```

Unesite podatke za admin korisnika (korisničko ime, email, lozinka).

3. Pokretanje aplikacije

Razvojno okruženje (Development)

Pokretanje backend servera

U terminalu s aktiviranim virtualnim okruženjem:

```
cd backend
.\.venv\Scripts\Activate
python manage.py runserver 0.0.0.0:8000
```

Backend će biti dostupan na: **http://localhost:8000**

Pokretanje frontend servera

U drugom terminalu:

```
cd frontend
npm run dev
```

Frontend će biti dostupan na: **http://localhost:5173**

Provjera rada

- **Aplikacija:** <http://localhost:5173>
- **Django Admin panel:** <http://localhost:8000/admin>
- **API dokumentacija:** <http://localhost:8000/api/>

Produksijsko okruženje (Production)

Backend (Gunicorn)

```
cd backend
.\.venv\Scripts\Activate
gunicorn --bind=0.0.0.0:8000 --timeout 600 backend.wsgi
```

Frontend (Build)

Prevođenje aplikacije za produkciju:

```
cd frontend
npm run build
```

Izgrađena aplikacija će se nalaziti u `frontend/dist/` direktoriju. Ove statičke datoteke možete poslužiti pomoću Nginx, Apache ili bilo kojeg drugog web servera.

Postavljanje s Nginx primjer:

```
server {
    listen 80;
    server_name vasa-domena.com;

    location / {
        root /path/to/frontend/dist;
        try_files $uri /index.html;
    }

    location /api {
        proxy_pass http://localhost:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /admin {
        proxy_pass http://localhost:8000;
        proxy_set_header Host $host;
    }
}
```

4. Upute za administratore

Pristup administratorskom sučelju

- **URL:** `http://localhost:8000/admin` (development) ili `https://vasa-domena.com/admin` (production)
- **Prijava:** Koristite superuser račun kreiran naredbom `python manage.py createsuperuser`

Administratorske mogućnosti

Administratori mogu:

1. **Upravljanje korisnicima:**
 - Pregledavati sve korisnike
 - Mijenjati uloge korisnika (student, moderator, admin)
 - Blokirati ili brisati korisnike
 - Odobravati nove moderatore
2. **Upravljanje objavama:**
 - Pregledavati sve objavljene skripte
 - Uređivati ili brisati neprimjerene materijale
 - Pregledavati komentare i ocjene
3. **Moderiranje sadržaja:**
 - Odobravati ili odbijati nove skripte
 - Provjeravati prijave neprimjerenog sadržaja

Redovito održavanje

Backup baze podataka

Lokalna SQLite baza (Development):

```
# Jednostavno kopirajte db.sqlite3 datoteku i media folder
$date = Get-Date -Format "yyyy-MM-dd"
Copy-Item backend\db.sqlite3 "backup_db_$date.sqlite3"
Copy-Item backend\media "backup_media_$date" -Recurse
```

Supabase baza (Production): - Supabase automatski kreira dnevne backupe - Ručni backup možete napraviti preko Supabase Dashboard → Database → Backups - Za Supabase Storage, koristite Dashboard → Storage → Backups

Restore baze podataka

Lokalna SQLite baza:

```
# Jednostavno vratite backup datoteke
Copy-Item "backup_db_2026-01-23.sqlite3" backend\db.sqlite3 -Force
Copy-Item "backup_media_2026-01-23\*" backend\media -Recurse -Force
```

Supabase baza: - Restore preko Supabase Dashboard → Database → Backups

Pregled logova

Django logovi se zapisuju u: - Standardni output terminala u development modu - U produkciji konfigurirajte logging u settings.py

Ažuriranje aplikacije

Kada se pojave nove verzije:

```
# 1. Povucite nove izmjene iz Git repozitorija
git pull origin main

# 2. Ažurirajte backend ovisnosti
cd backend
.\venv\Scripts\Activate
pip install -r requirements.txt

# 3. Pokrenite nove migracije baze podataka
python manage.py migrate

# 4. Ažurirajte frontend ovisnosti
cd ../frontend
npm install

# 5. Izgradite novu verziju frontenda
npm run build

# 6. Ponovno pokrenite servere
# Backend: gunicorn --bind=0.0.0.0:8000 backend.wsgi
# Frontend: refresh nginx ili deploy na hosting
```

Rješavanje problema

Provjera logova

Django:

```
# Pokrenite server s verbose outputom
python manage.py runserver --verbosity 3
```

Baza podataka: - Lokalno (SQLite): nema posebnih logova, greške se prikazuju u Django output-u - Produkcija (Supabase): logovi dostupni u Dashboard → Logs

Najčešći problemi

Problem: Ne mogu se povezati na bazu

```
# Lokalno (SQLite): Provjerite postoji li db.sqlite3 datoteka
Test-Path backend\db.sqlite3

# Ako ne postoji, pokrenite migracije:
python manage.py migrate

# Za Supabase (produkcija), provjerite:
# - DATABASE_URL environment varijablu
# - Status na Dashboard → Project Settings → Database
```

Problem: CORS greške - Provjerite ALLOWED_HOSTS u backend/settings.py -

Provjerite CORS postavke u `settings.py`

Problem: Frontend ne može pristupiti API-ju - Provjerite Vite proxy konfiguraciju u `vite.config.js` - Provjerite je li backend pokrenut na portu 8000

5. Deploy na cloud platformu (Render.com primjer)

Render je popularna cloud platforma za jednostavno smještanje aplikacija.

Priprema repozitorija

1. **Backend:** Uvjerite se da postoji `startup.txt` s naredbom za pokretanje:

```
gunicorn --bind=0.0.0.0 --timeout 600 backend.wsgi
```

2. **Frontend:** Dodajte build skriptu u `package.json` (već postoji):

```
"scripts": {  
  "build": "vite build --mode production"  
}
```

Postavljanje Backend servisa na Render

1. Prijavite se na [Render.com](https://render.com)
2. Kliknite **New** → **Web Service**
3. Povežite GitHub repozitorij `dunjamedurecan/Skriptomat`
4. Konfigurirajte postavke:
 - **Name:** `skriptomat-backend`
 - **Region:** Frankfurt (EU Central)
 - **Branch:** `main`
 - **Root Directory:** `backend`
 - **Runtime:** Python 3
 - **Build Command:** `pip install -r requirements.txt && python manage.py collectstatic --noinput && python manage.py migrate --noinput && python manage.py seed_roles && python manage.py seed_faculties && python manage.py setup_google_oauth && python manage.py create_oauth_app && python manage.py seed_courses`
 - **Start Command:** `gunicorn --bind=0.0.0.0:$PORT --timeout 600 backend.wsgi`
5. Dodajte **Environment Variables:**

```
# Django Settings
ALLOWED_HOSTS=skriptomat-backend.onrender.com,skriptomat-
fork.vercel.app,localhost,127.0.0.1
CORS_ALLOWED_ORIGINS=https://skriptomat-
fork.vercel.app,http://localhost:3000,http://localhost:8000
CSRF_TRUSTED_ORIGINS=https://skriptomat-
fork.vercel.app,http://localhost:3000,http://localhost:8000

# Database (Supabase PostgreSQL)
DATABASE_HOST=db.XXXXXXXXXXX.supabase.co
DATABASE_NAME=postgres
DATABASE_PASSWORD=your-supabase-database-password
DATABASE_PORT=5432
DATABASE_USER=postgres

# Django Core
DEBUG=False
SECRET_KEY=django-insecure-your-production-secret-key-minimum-50-
characters-long-here

# Email Configuration
EMAIL_HOST_PASSWORD=your-gmail-app-password-or-smtp-password
EMAIL_HOST_USER=your-email@gmail.com
ENABLE_EMAIL=True

# Frontend URL
FRONTEND_URL=https://skriptomat-fork.vercel.app

# Google OAuth
GOOGLE_CLIENT_ID=your-production-google-client-id-from-google-cloud-
console.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=your-google-client-secret

# PayPal Configuration
PAYPAL_CLIENT_ID=your-paypal-client-id
PAYPAL_CLIENT_SECRET=your-paypal-client-secret
PAYPAL_MODE=sandbox

# Supabase Configuration (Cloud Storage & Database)
SUPABASE_URL=https://your-project-id.supabase.co
SUPABASE_KEY=your-supabase-anon-public-key
SUPABASE_BUCKET=pdf-storage
SUPABASE_SERVICE_ROLE_KEY=your-supabase-service-role-key
```

6. Dodajte PostgreSQL bazu putem **Supabase**:

- Prijavite se na [Supabase.com](https://supabase.com) i kreirajte projekt
- U Project Settings → Database, kopirajte **Connection String** (URI format)
- **Važno:** Ne koristite Render PostgreSQL, već Supabase (besplatan tier ima automatske backupe i Storage za PDF-ove)

7. Pokrenite Deploy

Nakon deploya Render Environment Variables, dodajte:

```
SUPABASE_URL=https://your-project-id.supabase.co
SUPABASE_KEY=your-supabase-anon-public-key
SUPABASE_BUCKET=pdf-storage
```

Postavljanje Frontend servisa na Vercel

1. Prijavite se na [Vercel.com](https://vercel.com)
2. Kliknite **Add New** → **Project**
3. Importajte GitHub repozitorij
4. Konfigurirajte postavke:

- **Framework Preset:** Vite
- **Root Directory:** frontend
- **Build Command:** npm run build
- **Output Directory:** dist

5. Dodajte **Environment Variables**:

```
VITE_API_URL=https://skriptomat-backend.onrender.com/api
VITE_GOOGLE_CLIENT_ID=vas-production-google-id
VITE_PAYPAL_CLIENT_ID=vas-production-paypal-id
VITE_FIREBASE_API_KEY=vas-firebase-key
VITE_FIREBASE_AUTH_DOMAIN=vas-firebase-domain
VITE_FIREBASE_PROJECT_ID=vas-project-id
VITE_FIREBASE_STORAGE_BUCKET=vas-bucket
VITE_FIREBASE_MESSAGING_SENDER_ID=vas-sender-id
VITE_FIREBASE_APP_ID=vas-app-id
VITE_FIREBASE_MEASUREMENT_ID=vas-measurement-id
```

6. Kliknite **Deploy**

Nakon deploja Render Environment Variables, dodajte:

```
SUPABASE_URL=https://your-project-id.supabase.co
SUPABASE_KEY=your-supabase-anon-public-key
SUPABASE_BUCKET=pdf-storage
```

Pokretanje aplikacije

Render i Vercel će automatski preuzeti repozitorij, instalirati ovisnosti i pokrenuti aplikaciju. Nakon deploja:

- **Backend API:** <https://skriptomat-backend.onrender.com>
- **Frontend:** <https://skriptomat-fork.vercel.app>
- **Admin panel:** <https://skriptomat-backend.onrender.com/admin>

6. Pristup aplikaciji na javnom poslužitelju

Pristup aplikaciji

Javno dostupna Skriptomat aplikacija nalazi se na adresi:

<https://skriptomat-fork.vercel.app>

Korisnici mogu pristupiti aplikaciji putem bilo kojeg modernog internetskog preglednika (Chrome, Firefox, Safari, Edge).

Postupak korištenja

1. **Otvorite preglednik** i posjetite <https://skriptomat-fork.vercel.app>
2. **Registracija novog korisnika:**
 - Kliknite na “Registracija”
 - Unesite potrebne podatke: ime, email, lozinka, fakultet
 - Odaberite ulogu: Student ili Moderator
 - Možete se registrirati i putem Google računa
3. **Prijava postojećih korisnika:**
 - Kliknite na “Prijava”
 - Unesite email i lozinku
 - Ili koristite Google prijavu
4. **Korištenje aplikacije:**

- **Studenti** mogu: pregledavati skripte, preuzimati materijale, objavljivati svoje skripte, ocjenjivati i komentirati, slati donacije autorima
- **Moderatori** mogu: pregledavati i odobravati nove skripte prije objavljivanja, označavati neprimjeren sadržaj
- **Administratori** mogu: upravljati korisnicima, uređivati/brisati sadržaj, odobravati moderatore

Pristup administratorskom sučelju

URL: <https://skriptomat-backend.onrender.com/admin>

Pristup: - Samo korisnici s administratorskim ovlastima mogu pristupiti - Prijava je moguća s superuser računom - Za dobivanje administratorskog pristupa, kontaktirajte sistemskog administratora

Ograničenja javnog poslužitelja

Performanse

- **Render Free Tier:** Backend servis može “zaspati” nakon 15 minuta neaktivnosti. Prvo učitavanje može trajati 30-60 sekundi dok se servis ponovo pokrene.
- **Vercel:** Frontend je uvijek brz, ali ovisnosti prema backendu mogu biti sporije tijekom “hladnog starta”

Kapacitet

- **Supabase Free Tier:** Limit od 500 MB prostora za bazu, 2 GB bandwidth, 50 MB file storage
- **Upload fajlova:** Maksimalna veličina PDF skripte ograničena na 50 MB

Funkcionalnosti

- **Email obavijesti:** Ograničeno Gmail SMTP dnevnim limitom (~500 emailova/dan)
- **PayPal donacije:** U sandbox modu (testni način) - produkcijske transakcije nisu aktivne, ali transakcija je izvršena fiktivno s povratnom porukom o uspješnom plaćanju
- **Firebase Chat:** Ovisno o Firebase besplatnom planu (Spark plan)

Sigurnost

- **HTTPS:** Obje platforme (Render i Vercel) automatski pružaju SSL certifikate
- **CORS:** Konfiguriran samo za dopuštene domene
- **Rate limiting:** Preporučujemo implementaciju za zaštitu od zloupotreba

Dostupnost

- **Uptime:** ~95% dostupnost s povremenim pauzama zbog rada na stranici
- **Backup:** Potrebno je ručno preuzimanje backupa baze podataka
- **Skaliranje:** Omogućeno i fleksibilno

Dodatne napomene

Sigurnosne prakse

1. **Nikada ne commitajte .env datoteke** u Git repozitorij
2. **Koristite jake SECRET_KEY** vrijednosti u produkciji
3. **Postavite DEBUG=False** u produkcijskom okruženju
4. **Redovito ažurirajte ovisnosti** (`pip list --outdated, npm outdated`)
5. **Aktivirajte HTTPS** za sve produkcijske deploymente

Podrška

Za dodatna pitanja ili probleme: - Otvorite issue na GitHub repozitoriju:
<https://github.com/dunjamedurecan/Skriptomat/issues> - Kontaktirajte razvojni tim putem dokumenta CONTRIBUTING.md - Koristite integrirani chat za podršku unutar aplikacije

Verzija dokumenta: 1.0

Zadnje ažuriranje: 23. siječnja 2026.

Tijekom izrade projekta Skriptomat suočili smo se s mnogim izazovima, većinom tehničke prirode. Najveći izazov predstavljalo je učenje i snalaženje u različitim tehnologijama, kao i usklađivanje vremena potrebnog za izradu projekta s ostalim obavezama. Naučili smo se dobro organizirati, kao tim i samostalno. Komunikacija u timu uvijek je bila na visokom nivou. Tijekom izrade ovog projekta naučili smo koliko je bitna dobra i izravna komunikacija u timu. Bez problema izražavali smo ideje, poteškoće i zadatke koji se trebaju obaviti. Stekli smo znanja o Reactu, Django i ostalim tehnologijama koje su se koristile za izradu ovog projekta. Također, naučili smo izrađivati i UML dijagrame. Neki od problema na koje smo naišli tijekom izrade: preuzimanje skripti, plaćanje PayPalom, integracija chata, dolazak obavijesti. Uz puno truda uspjeli smo riješiti sve probleme. Implementirali smo sve funkcionalnosti koje su navedene u dokumentaciji. Nadogradnje projekta Skriptomat mogle bi biti: dodatni formati bilješki, dodavanje fakulteta i drugim državama povezanih s fakultetima u Hrvatskoj putem projekata (Erasmus...).

Rev.	Opis promjene/dodatka	Autori	Datum
0.1	Dodan predložak Wiki	Leonarda Laušić	23.10.2025.
0.2	Opisana arhitektura i dizajn sustava	Lorena Pratljačić	13. 11. 2025.
0.3	Dodan dijagram razreda	Dunja Međurečan	14.11.2025.
0.4	Dodana nadopunjena verzija dijagrama razreda	Dunja Međurečan	21.1.2025.
0.5	Dodani grafovi commitova	Dunja Međurečan	23.1.2025.
0.6	Napravljena tablica s aktivnosti korisnika	Dunja Međurečan	23.1.2025.
0.7	Opisano ispitivanje programskog rješenja	Lorena Pratljačić	23. 1. 2026.
0.8	Opisane tehnologije za implementaciju aplikacije	Lorena Pratljačić	23. 1. 2026.
0.9	Napisane upute za puštanje u pogon	Lorena Pratljačić	23. 1. 2026.

1. Programsko inženjerstvo, FER ZEMRIS, <http://www.fer.hr/predmet/proinz>
2. I. Sommerville, "Software engineering", 8th ed, Addison Wesley, 2007.
3. T.C.Lethbridge, R.Langaniere, "Object-Oriented Software Engineering", 2nd ed. McGraw-Hill, 2005.
4. I. Marsic, Software engineering book, Department of Electrical and Computer Engineering, Rutgers University, <http://www.ece.rutgers.edu/~marsic/books/SE>
5. The Unified Modeling Language, <https://www.uml-diagrams.org/>
6. Astah Community, <http://astah.net/editions/uml-new>

Dnevnik sastajanja

1. sastanak

- Datum: 10. listopada 2025.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * uspostavljena Whatsapp grupa svih članova > * prijedlog teme > * predstavljanje teme asistentici i demonstratoru > * uspostavljena Discord grupa svih članova

2. sastanak

- Datum: 17. listopada 2025.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * uspostava GitHuba i Wiki > * upoznavanje s git tehnologijom > * podjela uloga

3. sastanak

- Datum: 24. listopada 2025.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * odabir tehnologija

4. sastanak

- Datum: 31. listopada 2025.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * modeliranje funkcijskih i nefunkcijskih zahtjeva > * prezentiranje zahtjeva asistentici i demonstratoru

5. sastanak

- Datum: 7. studenog 2025.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * oblikovanje arhitekture sustava

6. sastanak

- Datum: 7. studenog 2025.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * implementacija osnovnih funkcionalnosti

7. sastanak

- Datum: 14. studenog 2025.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * osnovno ispitivanje implementiranih funkcionalnosti > * demonstracija inicijalne funkcionalnosti > * objavljivanje aplikacije na javnom poslužitelju

8. sastanak

- Datum: 12. prosinca 2025.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * analiza napravljenog > * planiranje sljedećeg koraka

9. sastanak

- Datum: 19. prosinca 2025.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * implementacija ključnih funkc. zahtjeva > * definiranje ispitnih slučajeva

10. sastanak

- Datum: 9. siječnja 2026.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * iterativna implementacija i kontinuirano ispitivanje > * analiza implementacije

11. sastanak

- Datum: 16. siječnja 2026.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * provođenje ispitivanja aplikacije > * dokumentiranje ispitivanja programske potpore

12. sastanak

- Datum: 23. siječnja 2026.
- Prisustvovali: L.Laušić, D.Međurečan, L.Pratljačić, D.Jovanović, J.Miličić, M.Smud
- Teme sastanka: > * završavanje implementacije > * završena projektna dokumentacija > * puštanje završne inačice na javni poslužitelj

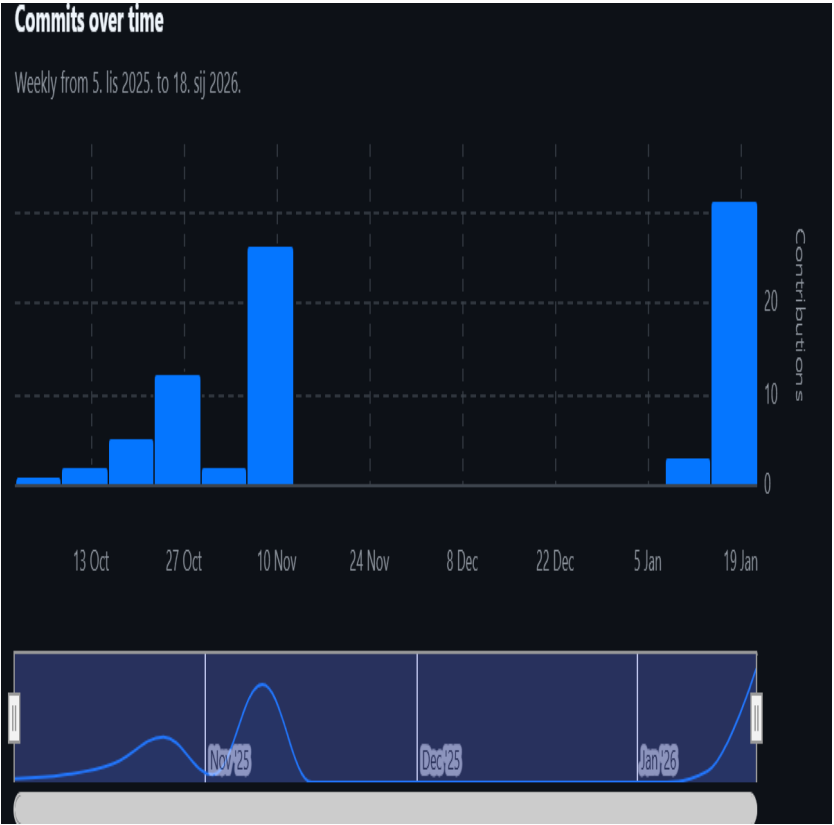
Tablica aktivnosti

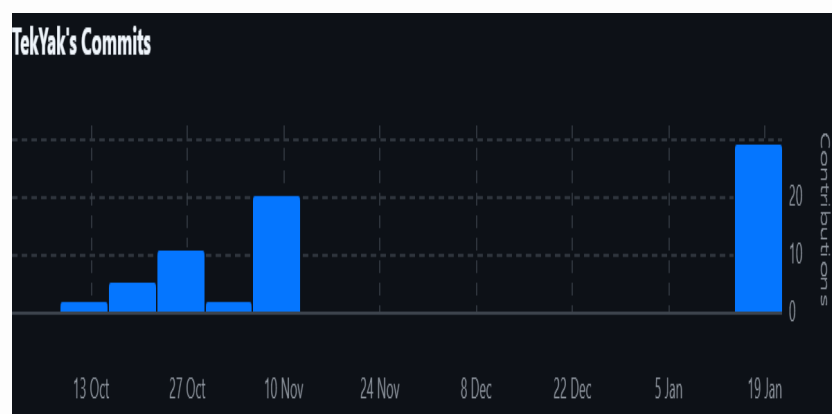
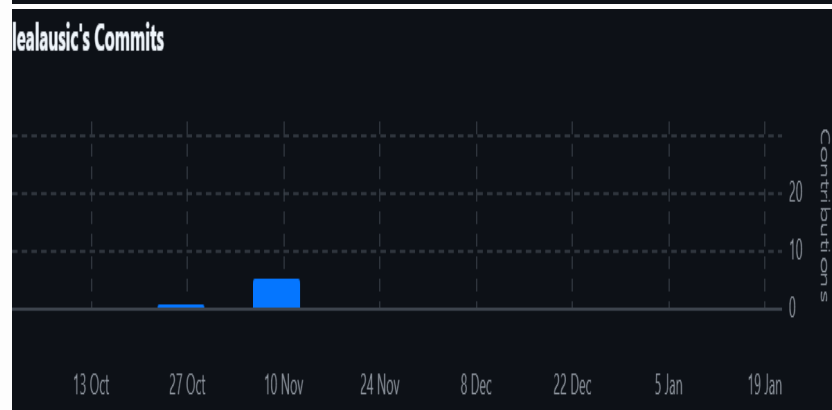
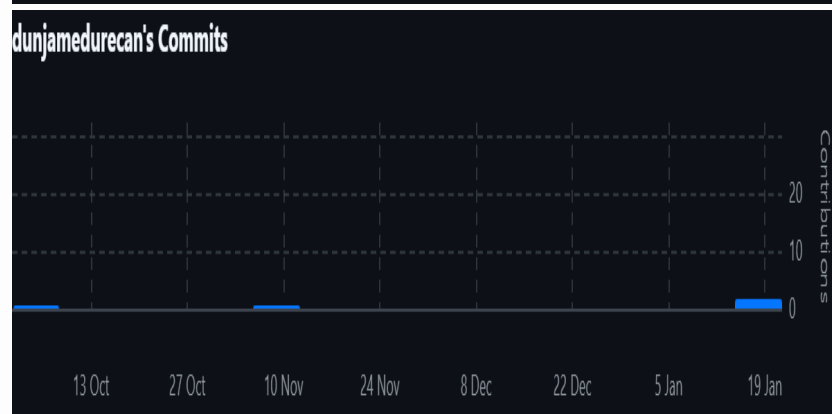
Aktivnost	D. Međurečan	L. Laušić	L. Pratljačić	D. Jovanović	J. Miličić	M. Smud
Upravljanje projektom	4	6	5	3	15	15
Opis projektnog zadatka	1	3	3	1	1	2
Funkcionalni zahtjevi	2	5	2			8
Opis pojedinih obrazaca		3	1			
Dijagram obrazaca		9				
Sekvencijski dijagrami		6				
Opis ostalih zahtjeva		3				
Arhitektura i dizajn sustava			8	10	5	6
Baza podataka			10	5	5	
Dijagram razreda	4					
Dijagram stanja		4				
Dijagram aktivnosti		3				
Dijagram komponenti		7				
Korištene tehnologije i alati	0.5					10
Ispitivanje programskog rješenja			5	4	5	6
Dijagram razmještaja		6				
Upute za puštanje u pogon			4			2
Dnevnik sastajanja		2				
Zaključak i budući rad		1				
Popis literature		0.5				
FE dio aplikacije	60			65	75	7
BE dio	100			80	75	100

aplikacije	100				80	75	100
Aktivnost	D.	L.	L.	D.	J.	M.	
Izrada baze podataka	8	Međurečan	L. Laušić	15	Pratljačić	5	Jovanović
Spajanje s bazom podataka	6			10	3	5	Miličić
Izrada prezentacije			2.5				Smud

Dijagram pregleda promjena

Prenijeti dijagram pregleda promjena nad datotekama projekta. Potrebno je na kraju projekta generirane grafove s githuba prenijeti u ovo poglavlje dokumentacije. Dijagrami za vlastiti projekt se mogu preuzeti s github.com stranice, u izborniku Repository, pritiskom na stavku Contributors.





Dio commit-ova nije vidljiv u GitHub statistici jer su napravljeni s email adresama koje nisu bile povezane s GitHub računima u trenutku commit-a. Stvarni doprinos svih članova tima

vidljiv je kroz commit history repozitorija.

Ključni izazovi i rješenja

Najveći izazov predstavljalo je učenje i snalaženje u različitim tehnologijama, kao i usklađivanje vremena potrebnog za izradu projekta s ostalim obavezama. Neki od problema na koje smo naišli tijekom izrade: preuzimanje skripti, plaćanje PayPalom, integracija chata, dolazak obavijesti. Uz puno truda uspjeli smo riješiti sve probleme.